中图分类号：TP391　　论文编号：153131326
学科分类号：520.6099　　密　　级：

天津理工大学研究生学位论文

面向博客管理的全文检索技术研究与实现

（申请工程硕士学位）

专业学位类别：工程硕士

工程领域：智能计算

作者姓名：孙黎月

指导教师：岳廷海　副教授

2017 年 12 月

Thesis Submitted to Tianjin University of Technology for the Master's Degree

Research and Implementation of Full Text Retrieval Technology for Blog Management

By

Sun Liyue

Supervisor

Yue Tinghai

Dec， 2017

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作和取得的研究成果，除了文中特别加以标注和致谢之处外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得 天津理工大学 或其他教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

学位论文作者签名：孙黎月　　签字日期：2018 年 1 月 12 日

学位论文版权使用授权书

本学位论文作者完全了解 天津理工大学 有关保留、使用学位论文的规定。特授权 天津理工大学可以将学位论文的全部或部分内容编入有关数据库进行检索，并采用影印、缩印或扫描等复制手段保存、汇编，以供查阅和借阅。同意学校向国家有关部门或机构送交论文的复本和电子文件。

（保密的学位论文在解密后适用本授权说明）

学位论文作者签名：孙黎月　　导师签名：

签字日期：2018 年 1 月 12 日　　签字日期：2018 年 1 月 12 日

摘要

伴随着互联网的急速发展，个人博客系统现在已经是一种传播个人想法与个人观点的极其重要的网络通信手段。个人博客系统使用户拥有了一个便利的可以交流讨论的平台。为了解决从海量数据中查找出用户想要的信息，全文检索系统就是在这种背景下应运而生。本论文主要研究了个人博客系统的设计与编程实现以及针对个人博客系统的全文检索技术，主要研究内容包括以下几个方面。

第一，通过对应用于文本分类的朴素贝叶斯模型的分析，本论文提出了一个能够对个人博客系统进行全文检索的算法，并通过实际的编程实验证实了该算法的可用性与正确性。该算法可用于中小型的全文检索系统的实现，同时可基于此算法开发定制个性化的搜索引擎。

第二，本论文应用 java 这一跨平台的高级程序语言来构建博客系统，使用先进的 MVC 设计模式，并在系统的编程实现中使用了流行的 SSH 三种框架，即 Struts、Spring 和 Hibernate。

第三，通过实验对比了由本论文提出的算法实现的全文检索与用 lucene 实现的全文检索的区别，得出利用该算法实现的全文检索方案可以随着用户点击量的增加而自适应的改变查询的排序结果，从而比用 lucene 实现的全文检索方案有更好的用户体验。

关键词：全文检索 朴素贝叶斯 中文分词 博客 费舍尔方法

Abstract

With the rapid growth of the Internet, the personal blogging system is now an extremely important means of network communication that communicates personal thoughts and personal perspectives. Personal blogging system allows users to have a convenient platform for the exchange and discussion. In order to solve the huge amount of data to find the information the user wants, full-text search system comes into being in this context. This dissertation mainly researches the design and programming of personal blog system and the full-text retrieval technology for personal blog system. The main research contents include the following aspects.

Firstly, through analyzing the naive Bayesian model which is applied to text categorization, this paper proposes an algorithm that can search the full-text of personal blog system. The actual programming experiment verifies the usability and correctness of the algorithm. The algorithm can be used for the realization of small and medium full-text retrieval system, and can be customized based on this model of personalized search engine.

Secondly, this thesis uses java as a cross-platform high-level programming language to build a blog system, uses advanced MVC design patterns, and uses three popular SSH frameworks, Struts, Spring and Hibernate, in system programming.

Finally, the difference between the full-text search realized by the algorithm proposed in this paper and the full-text search realized by Lucene is compared by experiments, and the conclusion that the full-text retrieval scheme realized by this algorithm can adaptively change the sorting result of the query as the user's hits increase, Resulting in a better user experience than the full-text search solution implemented with Lucene.

Key words: Full Text Retrieval, Naive Bayes, Chinese Word Segmentation, Blog, Fisher Method

目　　录

第一章 绪论 1
1.1 研究的目的及意义 1
1.2 研究内容与创新 2
1.3 论文结构 3
第二章 全文检索相关技术 5
2.1 全文检索 5
2.2 Lucene 5
2.3 中文分词 7
2.4 SSH 框架 8
2.4.1 Struts2 8
2.4.2 Spring 9
2.4.3 Hibernate 11
2.4.4 MVC 设计 13
2.5 本章小结 15
第三章 费舍尔方法用于全文检索 16
3.1 朴素贝叶斯分类方法 16
3.1.1 贝叶斯定理 16
3.1.2 概率论基础 16
3.1.3 朴素贝叶斯分类器 17
3.2 费舍尔方法 21
3.3 算法流程及实现 22
3.3.1 发表博文时进行预处理 24
3.3.2 提取查询词组 25
3.3.3 计算查询词组的词频 25
3.3.4 计算查询词组的后验概率 25
3.3.5 对后验概率进行加权平滑处理 26
3.3.6 合并查询语句的后验概率 27
3.3.7 运用费舍尔方法 28
3.3.8 逆序排序概率得到查询结果 28
3.3.9 在点击中持续学习 29

3.3.10 进一步的改进 29
3.3.11 举例说明 31
3.4 本章小结 36
第四章 系统的设计与实现 37
4.1 SSH 框架构建博客系统 37
4.1.1 系统需求分析 37
4.1.2 系统设计 39
4.1.3 编码实现 43
4.1.4 部署运行博客系统 43
4.2 费舍尔方法实现全文检索 43
4.3 Lucene 实现全文检索 46
4.4 两种实现方案的对比 48
4.5 本章小结 53
第五章 结论与展望 54
参考文献 55
发表论文和科研情况说明 56
致　　谢 56

第一章 绪论

在计算机网络迅速发展的今天，人们之间的相互交流、信息的传播逐渐依靠网络来进行。作为目前流行的网络交流方式，博客已经得到了广泛的发展。

除了能够自由地发表博文以外，博客还能使用户依照自己的想法来设计版式、定义功能。博客可以使用户在与其他网友交流的过程中累积成一个可以回顾的记录。加强个人博客系统的管理，这具有非常重大的意义。

随着互联网的快速发展和数据库技术的日益成熟，计算机已经可以存储大量的信息，如何在海量的信息中有效地查询想要的数据已成为人们越来越关心的问题[11]。人们从浩如烟海的数据中，快速、准确的获取自己最需要的信息却变得非常困难，此外国外的博客全文检索技术研究开始较早，已经推出了 Google 博客搜索引擎为代表的博客搜索引擎，技术发展也较为成熟[18]。然而这些博客搜索引擎在应用于个人博客系统时效果并不十分理想。

1.1 研究的目的及意义

随着网络的迅速发展，个人博客越来越流行，写博客已经成为一些网民日常生活的一部分。博客已越来越受到人们的重视。各大门户网站，比如腾讯、百度、网易都提供了自己的博客平台服务，并在其门户网页中占据重要位置。在博客中人们可以发布日常里自己和自己生活中发生的一些事情，同时自己的一些想法和自己想发表的一些知识笔记也可以记录在其中。通过博客可以将自己的观点与他人分享、交流，同时还可以交到很多有着共同爱好的朋友，而且甚至是分布在不同的国家。

博客在用户的日常生活中有着非常重要的用途，用户渴望把自己周边的奇闻轶事更新到博客与自己的朋友分享。

但是，现有的博客系统有不少的局限性：

1. 博客网页上的广告太多，这会影响博主的用户体验，也会影响浏览者浏览博客。

2. 博客系统对用户有很多限制，比如：在腾讯 QQ 空间超过一定字数的日志是不能发表的，上传到相册的图片下载时会丢失文件名，不能批量删除日志等。

3. 在现有的大多数博客系统中发表的博文等内容不能以文件的形式导出去。Google 的博客系统 Blogger 可以把用户发表过的博文以 XML 文件形式导出去，但是国内访问不了 Blogger。

4. 现有的博客系统不能保证用户数据的安全性和可靠性，可能会遭到黑客的攻击而导致泄露使用者的私密数据。

此外，对于个人博客系统来说，一个好的站内查询系统能快速、准确地使用户查询已经发表过的博文。全文检索功能可以方便用户管理其发表过的博文，是现在博客系统

必不可少的部分。然而，有的博客系统并没有全文检索功能或者提供了并不满足用户需求的全文检索功能[17]。比如，腾讯的 QQ 空间就没有实现搜索的功能，它的日志查询系统是不能使用的。

全文检索技术最早出现于 20 世纪 50 年代[19]。20 世纪 80 年代之后，国外陆续推出了许多具有全文数据库功能的全文检索软件，其中大部分产品都或多或少地具有数据库管理系统的功能，或者带有与通用的数据库软件的接口[19]。由于中文信息有其自己固有的特点，国内自主开发的中文全文检索软件系统较多[19]。

目前对全文检索技术的研究不少，常用的全文检索技术的实现方法主要有两种:

1. 利用数据库自带的索引来实现：现在的数据库管理信息系统，像 SqlServer，Oracle，DB2，Mysql 等都提供了索引，可以直接使用这些数据库系统自带的索引机制来实现全文检索。有的民营公司甚至使用数据库的语句 like+通配符的形式来实现全文检索技术。

2. 应用开源搜索引擎组件来实现：这个方法利用开源的搜索引擎组件来实现全文检索的功能，而且使用起来简单，不限制特定的数据库系统[12]。最流行的开源搜索引擎是 Lucene。

国外的全文检索系统尽管很早就得到使用，但对中国的博客系统很多不适用的地方[20]。

在目前的国内网络应用中，全文检索功能的使用并不理想，有的电子公文系统通过使用商用全文检索技术以获得查询支持，但需要较大的资金开销。因此，在针对站内的全文检索方面，还有很大的发展空间。

本课题正是在个人博客系统上实现对站内博文的全文检索。在实现个人博客的基础上，研究针对站内全文检索的实现。

1.2 研究内容与创新

本论文通过对应用于文本分类的朴素贝叶斯模型的分析，提出了一个能够对个人博客系统进行全文检索的算法，并通过实际的编程实验证实了该算法的可用性与正确性。

本论文的研究内容：

1. 使用 SSH 框架构建了 blog 系统。本论文采用流行的 SSH 框架来进行系统架构的开发。SSH 框架指的是 Struts2、Spring 和 Hibernate 三种框架。这三种框架各有各自的优点，共同协调配合来完成 java Web 的系统开发工作[14]。

2. 实现了两种全文检索方案。本论文使用了两种方式来实现全文检索：本论文提出的实现全文检索的方案；使用 lucene 技术来实现博客系统的全文检索的方案。

3. 两种全文检索实现方案的对比。本论文在自制的数据集上查询几组查询语句，对比两种方法实现全文检索的不同之处，并分析实验结果，查看是否有可改进的地方。

本论文的创新之处：

1. 提出了一种可以实现全文检索的算法。本论文通过分析应用于文本分类的朴素贝叶斯模型，提出了一个可以随着用户点击选择查询结果中的博文来进行持续学习的算法。该算法构建的模型可以随着后面的持续学习而自适应的动态改变用户的查询结果，从而

为用户提供更好的体验。

2. 解决目前大多数博客系统的限制，实现个性化的个人博客系统。本论文开发实现的个人博客系统可以创建多级目录，即目录的下面可以继续创建多个目录和多个文件。该博客系统可以实现无广告的网页，也可以自定义广告模块来满足用户的需求。此外，该博客系统可以发表不受字数限制的博文。

1.3 论文结构

本论文共有五个章节，这部分从整体上简要介绍论文的章节安排，并在宏观上把握论文的章节要点及各个章节之间的关系。本论文的结构图表示如下：

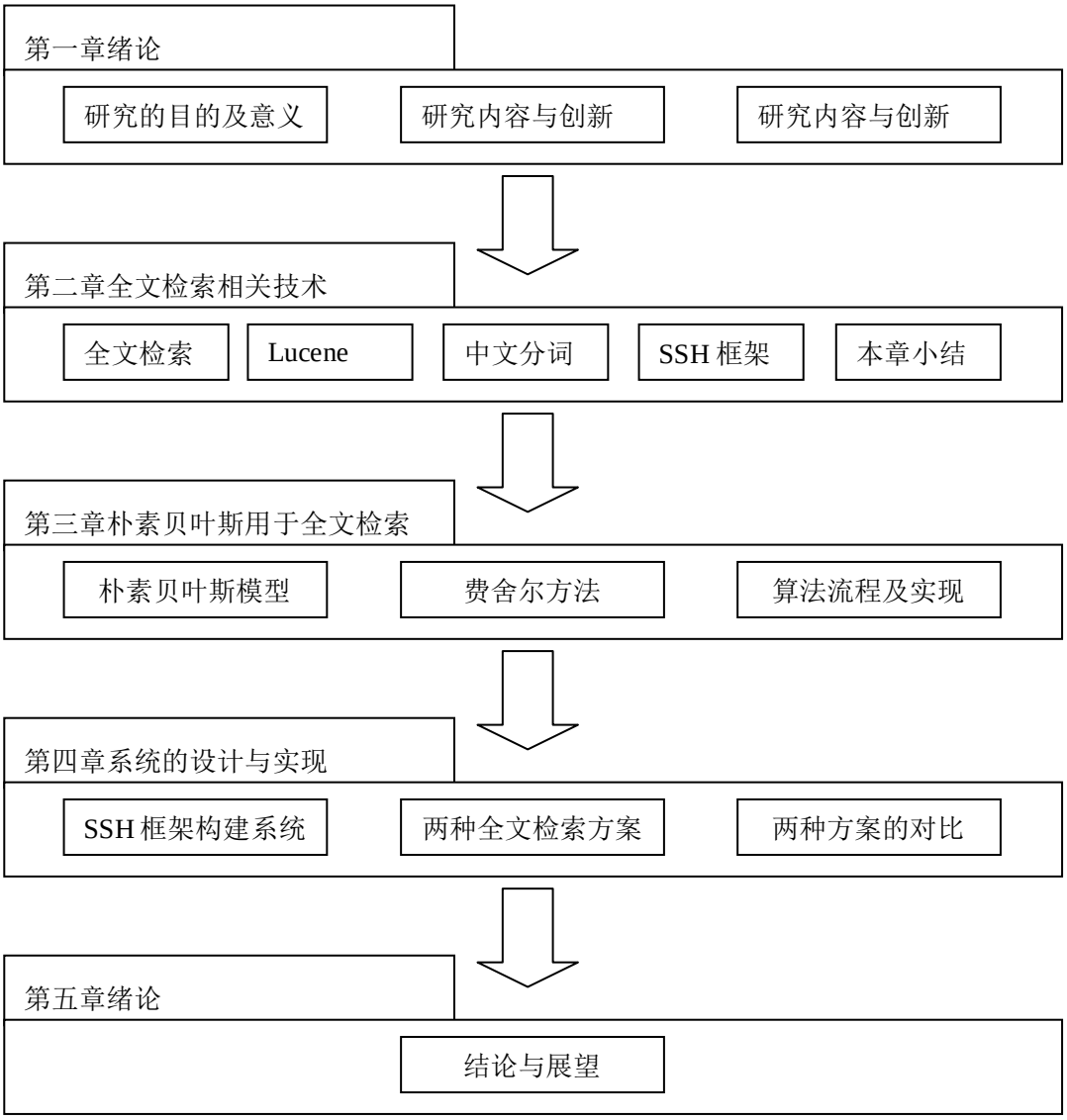


图 1-1 知识框架图

第一章：绪论，本章首先介绍了论文的研究目的及意义；其次阐述了本文的研究内

容与创新；最后针对论文的组织结构框架进行了阐述。

第二章：全文检索相关技术，主要讲述文章用到的理论知识：全文检索、Lucene、中文分词、SSH 框架及本章小结。

第三章：算法流程及实现，本章一共有三部分。首先介绍朴素贝叶斯模型及其它相关基础理论知识；然后讲述统计学家费舍尔及其开创的费舍尔方法；最后讲解算法流程，并详细介绍算法各个步骤的具体实现。

第四章：系统设计与实现，本章主要讲解如何用 SSH 框架构建博客系统，并在此系统上实现两种全文检索方式：用费舍尔方法实现的全文检索和用 Lucene 实现的全文检索。最后对比两种实现方案并进行总结。

第五章：结论与展望，本章总结论文的主要工作以及进一步的展望。

第二章 全文检索相关技术

本章讲述全文检索技术所用到的基础理论部分，主要内容为全文检索技术、Lucene、中文分词、SSH 框架和本章小结。本章详细的讲述了各部分内容。

2.1 全文检索

全文检索的核心技术是将源文档中的所有的基本元素的出现信息记录到索引库中[30]。全文检索是以各种计算机数据诸如文字、声音、图像等为处理对象，提供依照数据信息的内容而不是外在表现来实现的信息检索技术[2] [11]。

全文检索系统主要有五个核心模块：(1)建立索引；(2)增加索引；(3)优化索引；(4)处理查询返回结果；(5)显示查询结果列表[13]。

全文检索的过程如下：

1. 首先建立一个文本库，这个文本库用来存储用户可能查询的信息，在这些信息的基础上确立查询系统中的文本模型[11]。文本模型就是被查询系统认可的一种信息格式，一旦确立以后，不应该再对其进行大的变动[11]。

2. 建立索引，索引可以有效地提高信息查询的速度。采用哪种方式取决于查询系统的规模，大型的查询系统主要采用倒排序的方法来建立索引[11]。

3. 创建索引以后，就可以进行检索[11]。通常用户提交查询语句，系统对此查询语句分析后，利用文本操作进行处理[11]。

4. 对检索出的结果进行选择和排序，再将过滤排序后的结果返回给用户[11]。

2.2 Lucene

Lucene 是一个用 Java 语言编写的全文检索搜索组件，可以很容易地应用到各种系统中实现针对该系统的全文检索功能[1] [11] [29]。Lucene 仅仅提供了索引器和搜索器，因此只是一个全文搜索引擎的框架，而并不是一个完整的全文搜索引擎。即如果想使用 Lucene 的技术，就必须要在其基础上做一些必要的二次开发[3] [11]，将 lucene 框架加入到开发的系统当中。

Lucene 是 Java 领域的全文搜索的工具包，是非常流行、广泛被使用的工具包。在很多的全文检索功能的应用系统中，如果是使用 Java 来实现该系统的功能，都可能会看到 Lucene 的身影。美国在线 Apple、IBM 等公司在开发他们的系统当中都使用了 Lucene 的技术。Lucene 不仅仅可以对 HTML 数据、一般的文本文件的数据进行搜索，还可以支持搜索 Excel 和 PDF 等格式的数据。Lucene 会对这些数据建立索引并且进行搜索。

Lucene 是 Hibernate Search 底层使用的技术。Hibernate Search 是 Hibernate 底层实现

全文检索的技术。Hibernate 在底层上没有使用自己的全文检索技术，而是直接依赖于Lucene。随着时间的发展，Lucene 现在不仅在 Java 领域提供全文检索的技术，而且在.Net 和 C 语言方面也提供了类似的一些帮助。由于本论文使用 Hibernate Search 来实现 Lucene 的全文检索功能，故在此简单介绍一下 Hibernate Search 的特点。Hibernate Search 是在 Hibernate 当中提供全文索引的一个组件，利用 Hibernate Search 就可以对 Hibernate 中的数据进行全文索引的查找和搜索。同时，Hibernate Search 还支持索引数据的自动更新，比如使用 Hibernate Search 来保持、删除和修改数据以后，这些被变动或增加的数据都会同步地进行相应的索引数据的更新。此外，Hibernate Search 支持众多的搜索方式，最主流的是支持使用关键词进行搜索，还支持通配符方式的搜索，而且也支持近义词或者同义词的搜索。如果要搜索请求的数据是海量的，Hibernate Search 还可以通过多台机器来提供集群搜索。在 Hibernate Search 当中，它还支持对 Lucene API 的直接调用，这是因为它的底层使用的就是 Lucene 技术。当 Hibernate Search 不能满足用户的需求时，它可以让开发人员通过它对底层的 Lucene API 进行直接的调用。

Lucene 功能极其强大，但从本质上来说，主要包括两大块，即创建索引部分和查询检索部分两大部分[11]。下面结合上述的论述给出 Lucene 的逻辑结构图，如图 2-1 所示：

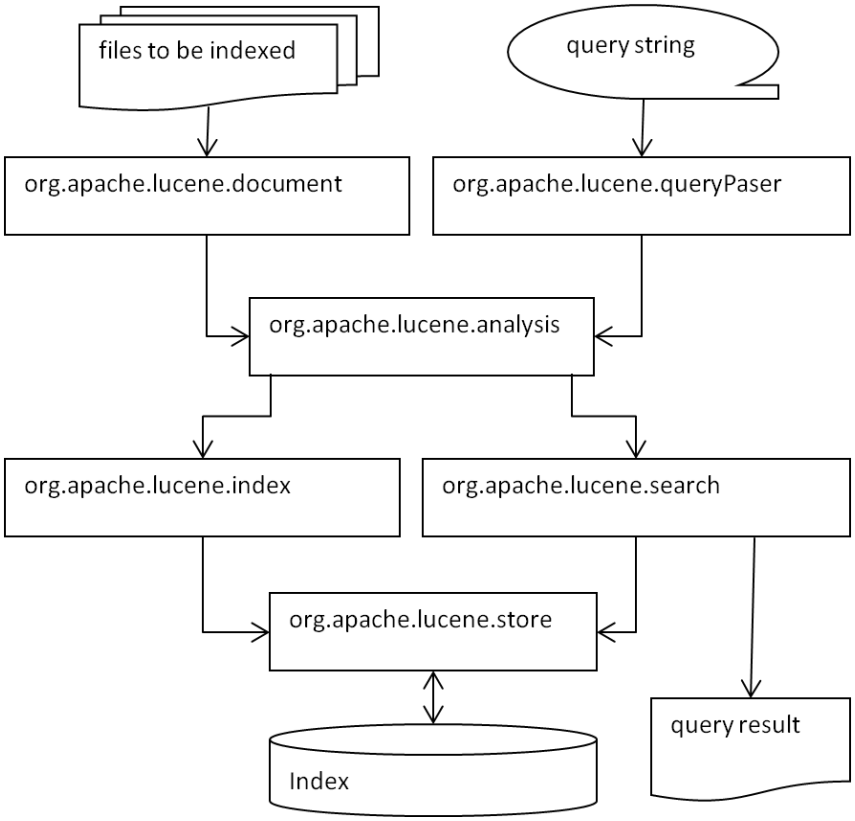


图 2-1 lucene 的逻辑结构图

传统的查询技术查询速度比较慢，尤其在处理海量数据和模糊查询时有着明显的不足[11]。Lucene 通过使用特殊的索引机制实现了传统查询不擅长的全文索引技术[11]。表 2-1 对比了 Lucene 和其它全文检索系统的不同之处。

表 2-1 lucene 和其他全文检索的区别

	Lucene 全文检索	其他全文检索系统
索引方式	可以进行增量式的索引，也可以进行批量式的索引	只支持批量式的索引
索引内容	Lucene 的文档是由多个字段组成的，能够控制查询哪个字段，和不查询哪个字段[15]。	通常检索了整个文档
语言分析	支持非英文语言，可以过滤忽视不必要的常用词，例如“的”字，可以进行语法分析，例如可以将 jumped 和 jumping 都当作 jump 处理	没有通用接口
查询分析	可以自定义查询语法的规则，自定义关键字的+，-，与，或关系	

虽然 Lucene 与数据库有许多相似的地方，但全文检索并不等同于模糊查询[4]。有数据库开发经验的程序员都会知道，在系统运行过程中，如果过多使用 Like 关键字来实现对数据库的查询，系统运行的效率往往会不高。

2.3 中文分词

中文分词就是把中文的句子切分成有意义的词组[17]。在英文序列中，由于字与字，词与词之间会有较为明显的空格，而且英文每个单词都有独立的意思，所以英文分词完全可以使用空格作为词与词之间的分隔符，从而很快地就可以完成分词的操作[23]。对于英文单词的提取，还有一项可能要做的工作，即利用某种词干提取算法去除掉单词的后缀。词干提取算法试图将单词转换成对应的词干，也就是动词原形[8][16]。例如：将单词“indexing”变成“index”。这样，人们在搜索单词“index”时同样也会得到包含单词“indexing”的文档[16]。要做到这一点，我们可以在检索文档期间提取单词的词干，也可以在搜索查询期间提取单词的词干[23]。与英文相比，中文分词更为复杂，一句话不同的切分可以得到不同含义的词组，所以不可能使用简单的分隔符完成分词操作。

从工程技术的角度来看，中文分词的基本机制有两种：一种是采用词库，即把很多的中文词汇放到一个库里保存，词库里有所有的常用词，然后在分词的时候通过与库里词汇一个一个的匹配比对，知道哪些词汇是有意义的，比如“中国”是一个词汇，“中国人”也是一个词汇；第二种方式是自动切分，比如先一个一个汉字的切分，然后再按照相邻的两个两个汉字去切分。这种方式执行效率低，会产生很多没有意义的词组。这是最基本的两种机制。这两种分词的机制各有优点与不足，如表 2-2 所示：

表 2-2 两种分词的机制

	自动切分	词库表切分
实现	实现简单	实现复杂
查询	增加了查询分析的复杂程度	适用于实现比较复杂的查询语法规则
存储效率	索引冗余大，索引几乎和原文一样大	索引效率高，为原文大小的 30%左右
维护成本	无词库维护成本	词表维护成本高，还需要包括词频统计等内容

本论文在设计博客系统时使用 SSH 框架，并需要用到 lucene 组件。这样在系统进行中文分词时要用到 lucene 的分词机制来配置 SSH 架构。这里简单介绍一下 lucene 中文分词器。由于 Lucene 刚开始时主要是针对英文检索开发的，所以其默认的分词器主要还是标准的分词器，采用直接通过空格分离的方法完成对句子的切分，很容易就能完成所需的功能[17]。但对于中文来说，其官方只提供了在标准分词器基础上完成的中文单字以及双字分词两种模式的方法，虽然在一定程度上是能够使用，但并不能很好实现用户的需求[17]。下面将介绍本系统使用的庖丁解牛分词器

庖丁解牛分词器（PaodingAnalyzer）是基于 Lucene 的比较常用的一种中文分词器[17]。该分词器可以对未知的词汇进行合理解析，对于词典中没有的词进行二元分词，采用完全的面向对象设计，在中文分词领域基本满足要求[17]。

由于本系统使用庖丁解牛分词器，故不再介绍其他几种中文分词器。

2.4 SSH框架

SSH 框架是当前比较流行的 Web 开发框架，使用了 Struts2、Spring 和 Hibernate 三个框架来进行无缝地开发 Java Web 系统。其中使用 Struts2 负责 MVC 的分离，使用 Hibernate 对持久层提供支持，使用 Spring 管理 Struts 和 Hibernate。

采用 SSH 框架开发模型，有利于团队成员并行工作，提高了开发效率。

2.4.1 Struts2

Struts2 是实现了典型的 MVC 模式的 Java Web 开发框架。Struts 利用 Servlet 作为控制器，为视图和模型搭起了一个联系的桥梁。

Struts2 可以用来实现 MVC 设计模式，下面介绍 Struts2 和 MVC 对应的各个组件：

模型部分：Java Bean、Service Class，用来封装业务数据、业务逻辑和程序运行状态。

视图部分：JSP 文件，用来生成动态网页。ActionForm Bean 是一种 Java Bean，在

Struts2 中可以用来对表单数据进行操作。JSP 文件可以读取 ActionForm Bean 中保存的数据。

控制部分：包括 ActionServlet 类、Action 类以及 struts-config.xml 等内容。ActionServlet 对象负责接收用户的请求，并根据 struts-config.xml 文件中的配置信息，将用户请求、表单中的数据传递给相应的 Action。Action 类用于对程序运行流程的控制，负责处理 Servlet 传递过来的用户请求。struts-config.xml 负责配置用户请求和 Action 的映射。

Struts2 的基本工作流程为：

1. Web 服务器启动，初始化 ActionServlet，读取 struts-config.xml 文件中的配置信息，并对这些信息作相应的分发处理[21]。

2. 用户发出一个请求后，ActionServlet 接收到这个请求，查看和用户请求匹配的 Action 映射对象是否存在[21]。

3. Action 调用 JavaBean，并结合 ActionForm 中的数据，对用户的请求作出处理。

4. Action 处理完毕，返回一个 ActionForward 对象给 ActionServlet。这个 ActionForward 对象指定了转发的目标对象，可以是某个 JSP 文件，或者一个新的 Action 对象。

5. ActionServlet 根据 ActionForward 的指向，将对用户请求处理的结果传送给相应 JSP 文件，或者转发到一个新的 Action 对象，继续从第 2 步进行处理[21]。

2.4.2 Spring

Spring 作为当今最为流行的开源框架，它采用了分组件的设计模式，开发者可以根据自己的需求选择所需要的框架，此外，它还可以为程序员提供集成的框架。

Spring Framework 是轻量级的 Java EE 开发框架，由 Rod Johnson 创始而来的。Spring 发展到现在，其最新的版本是 5.0.3，集成的开发框架已经十分庞大。在最核心的 Spring Framework 框架中，Spring 分成了六个模块：DAO、ORM、JEE、Web、AOP 和 Core。如下图 2-2 所示：

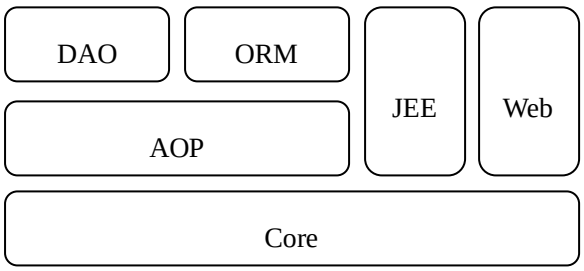


图 2-2 Spring 的六个模块

在这六个模块中，Core 模块是 Spring 中核心的基础模块之一。Core 模块利用控制反转和工厂模式可以对 Spring 中要用到的 Bean，即类对象进行生产和相应的组装。AOP 模块也是 Spring 中的核心模块，为进行 AOP 方面的开发提供全面的支持。AOP 即面向方面或面向切面的编程。DAO 模块提供对使用 DAO 模式开发的一些支持，同时这个模

块还提供了封装 JDBC 操作的相应的工具类。ORM 模块提供对其他 ORM 框架进行整合支持的工具类，比如要把 Hibernate 与 Spring 进行整合就需要用到 Spring 当中的 ORM 模块当中的一些类。JEE 模块是一个包含了许多杂项的模块，主要提供了对 Java EE 开发当中需要的一些支持，比如为整合 JMS、JMX 以及发送邮件等服务提供相应的支持。Web 模块对整合其他的如 MVC 模式的框架提供支持，比如在 Spring 中整合 Struts 框架可能就要用到 Web 模块当中的一些类。同时，在 Web 模块当中，Spring 也有自己的一个 MVC 框架 Spring MVC。在 Spring 的六个模块中，不同的模块放在不同的 JAR 文件中，需要用到哪个模块就使用哪个模块对应的 JAR 文件。还可以直接使用一个 Spring.jar 文件，这个 JAR 文件几乎包含了六个模块所用到的类和接口等文件。

在 Spring 当中，有两个核心的技术：IoC 容器和 Spring AOP。

1．IoC 容器

容器可以理解成能够包含其他对象的对象，即容器自身也是一个对象，但是这个对象很特殊。容器的特殊在于它能够包含其他对象，非常像使用高级编程语言做开发中的 ArrayList 和 Vector 等对象。容器可以划分为好几种：有运行 Servlet 和 JSP 的 Web 容器；有能够运行 EJB 的 EJB 容器。而在 Spring 中也有一个容器，即 IoC 容器。IoC 容器所管理的对象都应用了 IoC 的设计模式。IoC 是英文 Inversion of Control 的首字母缩写，中文意思为控制反转。IoC 设计模式又可以称为 DI。DI 是英文 Dependency Injection 的首字母缩写，中文意思为依赖注射。

简单来说，IoC 模式是这样运行的：当一个对象用到其他类型的对象时，通常是通过关键字 new 来生成一个不同类型的对象。当使用 IoC 模式以后，这个过程就可以简化。当需要用到另一个不同类型的对象时，通过 Spring 的配置，然后把要用到的对象使用 set 方法注射到当前的类型对象当中即可。将来不想使用此依赖对象或者是想使用另外的依赖对象，这时改变一下 Sping 的配置文件即可。

2．Spring AOP

AOP 是英文 Aspect Oriented Program 的首字母缩写，中文意思是面向方面或面向切面。AOP 是对 OOP 的一种补充。Java 和 C++都是面向对象的高级编程语言，都使用了面向对象的编程思想。在这种编程思想当中都有封装、继承和多态的概念，利用这些概念可以很好的建立一种具有层次的关系结构。这种关系结构非常适合项目的模块化开发，即项目中的类和模块可以重复的使用。但是，利用 OOP 去编程的时候，有些情况下是不适用的。当利用 OOP 去实现一些逻辑时，这些逻辑有可能需要一些非核心的功能：日志的记录、事务的处理和权限的检查等。这些都和开发核心功能关系不大，但是又是当前开发的应用中必不可少的功能。OOP 没有特殊的组织功能来管理这些非核心应用，只能在各个核心模块当中重复的实现这些非核心应用。AOP 可以将这些非核心的应用集中放到一个专门的模块中去处理，并将它们像类一样地得到重复使用。这样，开发人员在编写核心模块时就不需要去考虑日志记录和事务处理等功能。此时，AOP 已经将这些功能交给另外的模块去处理，而这个模块就是 AOP 模块。

AOP 有一些核心概念：连接点、切入点、通知、切面、目标对象、AOP 代理和织入。

连接点（Join Point）：当使用 AOP 时，在某些方法执行前和执行后需要加上一些其他代码。比如，有一个核心逻辑代码来输出学生的姓名，在这个方法的执行之前和执行之后都可以加上日志处理。添加日志处理的代码并不是在核心逻辑代码中写的，而是通过 AOP 来加进来的。能加入日志处理代码的位置就被称为连接点。

切入点（Point Cut）：通过使用一些表达式来匹配一些连接点，从连接点中选择开发过程中需要用到的几个连接点，并在这几个连接点上进行操作。这些表达式通常是正则表达式。

通知（Advice）：指的是在感兴趣的连接点上要执行的增强的方法、增强的内容、增强的代码。比如添加的日志记录和事务处理就是通知。

切面（Aspect）：指的是通知和切入点的组合。定义一个表达式来拦截一些方法，并要对这些方法执行一些处理，这两个组合在一起就是切面。

目标对象（Target Object）：指的是原始的对象，比如有一个 Student 类，这个类里面有些方法。现在需要对这个类对象进行拦截处理，在方法之前和之后进行增强。这个 Student 对象就是目标对象，即会被切面处理的对象。

AOP 代理（AOP Proxy）：在 Java 中实现 AOP 一般都会使用 Java 中的代理模式。这样会生成当前类的一个代理对象，AOP 代理指的就是所生成的代理对象。

织入（Weaving）：原来的代码是没有日志处理的，也没有事务处理。现在通过 AOP 添加了日志处理和事务处理，这个添加的过程就叫做织入。

在 Spring 当中还可以使用 AspectJ 5。AspectJ 5 是另外的一种 AOP 的实现，它与 Spring 也有一定的整合。AspectJ 的 AOP 实现可以使用注解，可以很方便地设置 AOP 的一些通知、切入点以及 AOP 的拦截等。过去开发人员都要写一些相应的代码，而且类、方法才能实现 AOP，现在使用注解就不受此限制。Spring 也可以使用注解，Spring 的注解来源于 AspectJ 5，它本身没有自己的注解，而是直接使用 AspectJ 中的注解。

此外，Spring 提供对 Hibernate 开发的支持：

1．Spring 可以简化 Hibernate 中 SessionFactory 对象的获取。当使用 Hibernate 来开发系统时，一定需要用到 Hibernate 的 Session 对象，而 Session 对象是需要 SessionFactory 对象来生成的。通过把 SessionFactory 对象注射到 DAO 类当中，就可以在 DAO 类当中使用注射过来的 SessionFactory 对象来获取 Session 对象。

2．使用 Spring 提供的 HibernateTemplate 类和 HibernateDaoSupport 类来简化 Dao 实现类的编写。

3．使用 Spring 的声明性事务来管理其他事务。比如，HibernateDao 当中有一些方法需要事务处理，通过 Spring 当中的注解就可以很方便的实现声明性的事务管理。

2.4.3 Hibernate

Hibernate 是一种强大的对象/关系映射的框架，允许通过对持久化对象进行操作来完成对底层数据库中数据的操作[21]。

1．框架

框架（Framework）是整个或部分系统的可重用设计，表现为一组抽象构件及构件

实例间交互的方法。使用框架来开发系统相当于有一个现成的半成品，然后在这个半成品上进一步的添砖加瓦，而不再从头开始。

利用框架可以更好、更快地开发一些需要用到的应用系统。总的来说，框架是一个应用程序开发的半成品，然后它在这个半成品里面针对很多人开发的应用系统的共性提供了可重用的公共模块。通过使用框架就可以使用它提供的公共模块，同时会依据该框架的一些配置文件让这个框架更适于当前系统的开发与使用。

ORM 框架是一种持久层的框架。字母 O 指的是对象（Object），字母 R 则指的是关系型数据库系统（Relational Database），字母 M 指的是映射（Mapping），翻译成中文就是对象关系型数据库映射。ORM 框架可以帮助开发人员对数据库上的操作变成对象化的操作。比如有一个 USER 表，该表有三个字段：id，name 和 age。当使用 ORM 框架去处理这个表的数据时，ORM 框架不是以表的形式来处理数据，而是以对象的形式。使用以对象的形式来处理数据会把 USER 表看作一个类 User，而且这个类有三个属性，一一对应表 USER 的三个字段。使用 SQL 语句读取的是表中的一条记录，而在 ORM 框架的映射下，读取的是一个类 User 的对象 user。使用 ORM 框架后就可以直接对对象进行操作，因为它已经完成了一个表与类的映像。当向 USER 表添加一条记录时，借助 ORM 框架生成 User 类的一个对象，并对该对象的各个属性赋值后保存，ORM 框架就可以自动地在 USER 表中添加一条记录。当想删除 USER 表中的一条记录，借助 ORM 框架找到这条记录所对应的对象，然后通过框架的 API 调用一个方法就可以把这个对象删除掉，进而 USER 表中该对象对应的记录也就随即不存在了。当想修改 USER 表中的一条记录，先找到该记录所对应的对象，然后修改该对象的属性值即可。 使用 ORM 框架以后，对数据库的操作都是以对象的形式来进行的。传统的对表的 SQL 操作已经转化成一种面向对象的思路去进行，这样在数据操作上更简单，不需要开发人员在底层的数据库上去操作，而且可以在设计上进行统一，可以在底层数据库设计上也采用面向对象思想。常见的 ORM 框架有 Hibernate，iBATIS 等，但 Hibernate 框架最有名而且使用最广泛。

2．持久层

随着开发的应用系统复杂度的提高，为了更好的开发系统，通常会把系统的开发划分为多个层次：可以划分为四层，也可以划分为三层，还有划分成五层。一般会把系统的开发划分成四层：数据层、持久层、业务逻辑层和表现层。各个层次有着不同的功能。

持久层协助开发人员把数据保存在数据库，或者是修改、删除以及读取数据。ORM 框架是持久层的基础，JDBC 也是持久层的基础。

Hibernate 框架是非常流行、非常成熟、性能稳定、参考文档齐全的开源产品。

自 Hibernate3 发布以来，其产品线愈加成熟，相继出现了 Hibernate Annotations、Hibernate Search 和 Hibernate EntityManager 等一系列组件。Hibernate 框架已经变成了一个大家族，而不再是独立的一个组件，并且拥有若干个小的模块。Hibernate 组件之间的关系如图 2-3 所示：

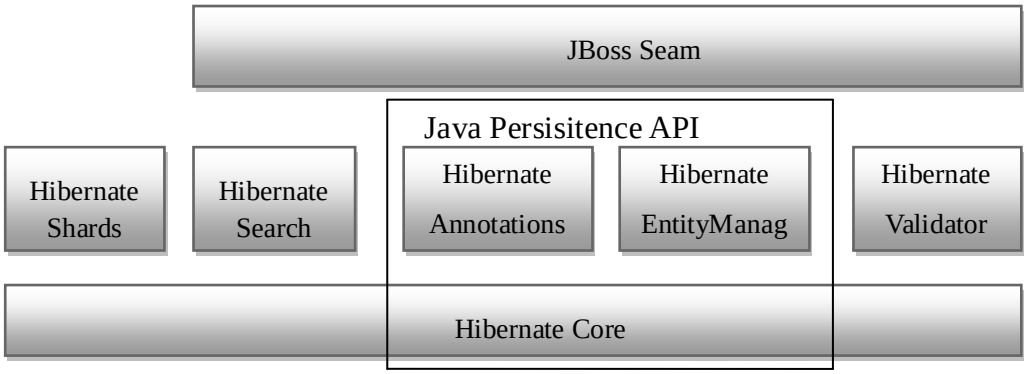


图 2-3 Hibernate 组件之间的关系

Hibernate Core 是 Hibernate 中最基础的部分，其他的组件都是在它的基础之上。它提供的 ORM 框架核心功能可以帮助开发人员大大节省开发持久层的时间。

Hibernate Annotations 是帮助开发人员使用注解来完成持久化类和数据库中表之间的映射。Hibernate 可以使用配置 XML 持久化映射文件的方式来实现映射，也可以使用 JDK5.0 以后提供的注解的方式来实现映射，而使用后一种方式来实现映射更加简洁，也提高了开发的效率。

Hibernate EntityManager 是为了兼容 JPA 而提供的一个组件。Hibernate Core、Hibernate Annotations 以及 Hibernate EntityManager 共同实现了 JPA（Java Persistence API）接口，并提供了兼容 JPA 的程序设计。JPA 是 Sun 公司提供的用来完成持久化操作的一套接口。

Hibernate Search 是用来实现全文索引的一个组件。它在底层使用了 Lucene 技术，可以提供快速的数据库数据的全文检索。

Hibernate Shards 是用来做数据库分区的。当一个应用系统保存的数据是海量的时候，这些数据都要保存在数据库中。只使用一个数据库来保存这些数据是无法胜任的。这时就要用到数据库分区的概念，把这些海量数据存储到若干个数据库中，Hibernate Shards 就是用来帮助数据库分区的一个组件。

Hibernate Validator 是用来对数据进行校验的一个组件。使用该组件可以对添加到数据库中的数据进行校验。其功能与 Strust2 的校验框架很接近，比如校验数据不能为空以及长度上的限制等等。

2.4.4 MVC设计

MVC 名称的来历是指三个单词的首字母缩写，即 Model、View 和 Controller。这三个单词翻译成中文就是模式（Model）、视图（View）和控制器（Controller）。MVC 模式是一种架构模式，实际上它在底层上是几种原始模型的结合使用[26]。MVC 模式综合运用了多种设计模式，包括观察者（Observer）、策略（Strategy）和合成（Composite）等模式，由 Trygve Reenskaug 在 1979 年提出和使用[26]。

MVC 模式不是随着 Java Web 的开发而产生的一种模式，应该说它是一种很古老的模式。准确的说，MVC 模式应该是上个世纪七十年代就被人们提出来的，当时主要应用于图形用户界面的系统设计上面。在这种架构的情况下，它把系统分为三个部分：控

制器部分、视图部分和模型部分。这三个部分做不同的功能，综合起来使用，然后实现整个系统：可以是个 Java Web 的系统，也可以是个图形用户界面的系统。这三个部分的结合使用如下：视图主要用来显示；模型主要完成数据的保存、读取和业务逻辑的运算；控制器主要起到协调和控制的功能。

为了更好的介绍 MVC 模式，这里举一个例子：假如现在采用 MVC 模式开发一个股票系统，这个股票系统是一个图形用户界面的系统。在这个股票系统当中，用户可以查看股票的价格。由于这是一个图形用户界面系统，在查看股票价格的效果上可以在图形用户界面上有一个窗口。而在这个窗口中股票价格的显示可以有若干种形式：可以是简单的数字列举，比如中国石油 25 块，中海油 15 块；还可以是以曲线图的形式来查看，比如想查看股票最近的走势；还可以是以比较的方形图的形式，比如把中石化和中海油以及很多其他股票的价格比较一下。也就是说，用户看到的最终的效果不是一种，而是很多种，而且是可以改变的。根据用户的不同请求，用户可以看到不同的显示效果。这个其实就是有多种视图的系统。在这个系统当中，用户通过鼠标点击按钮发出请求，这个时候接受请求的系统当中会有一个专门的控制器部分。控制器部分接受到用户的请求，判断出用户到底需要哪种效果的视图，并选择用户想要的显示效果的视图，把最终的显示结果显示给用户。最后，用户会看到非常漂亮的显示结果。然而，最后的显示结果是需要数据来支持的，即股票价格。股票价格是单独的另外一个模块，它是放在了模型部分当中的。

从上面的例子中可以看出来，这三个部分各司一职，但是又是综合作用，最后得到用户想要的显示效果。控制器部分主要完成控制的功能，接受用户的请求，返回用户想看到的效果；视图只做显示的功能，用户想要什么样的效果就显示给用户什么效果；模型是保存数据的部分。从而三者结合在一起。这是图形用户界面的一个示例，当将 MVC 模式用在 Java Web 中时也是类似的。在 Java Web 中，用户通过浏览器发出请求，该请求会被传给控制器。控制器接受到用户的请求后，判断出用户想要做什么。控制器知道用户的请求是什么含义之后，会调用相应的模型，即 Java Bean 部分，来完成相应的逻辑运算或者是数据的读取与保存。最后，控制器会把计算的结果通过 JSP 页面显示给用户。这就是 Java Web 中 MVC 模型的一般过程。实际上，MVC 模型在 Java Web 开发中的使用是非常广泛的。在这种架构之上，后来又引申了很多现成的框架，比如 Spring MVC，Struts 和 Web Work。视图、控制器和模型之间的相互关系如图 2-4 所示：

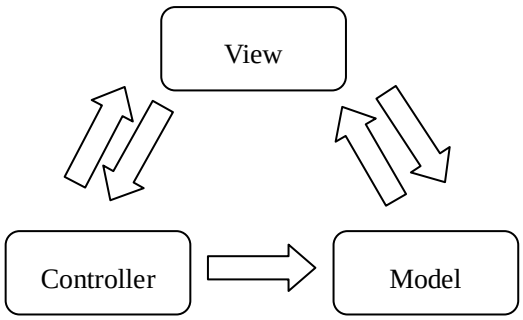


图 2-4 Mode View Controller 关系图

使用 MVC 来进行 Web 项目的开发，系统的分工也很明确[26]。一般情况下，视图部分是由美工或者网页设计人员来开发的，控制器部分由 Java 开发人员进行开发，模型部分由 Java 高级开发人员、领域专家和数据库管理员等共同完成[26]。

MVC 的优点：

1. 各司其职，互不干涉[26]。MVC 模式已经很明确的给系统分了三个部分。每个部分完成特定的任务，交给特定的工作人员去做。这样，开发系统的工作人员的工作就不会互相干扰。

2. 有利于开发中的分工[26]。在一个很大的系统开发中，不是一个人独自完成所有的工作，而是几百人甚至上千人共同合作完成的。现在开发软件系统都是很多不同种类的工作人员协作完成的。使用了 MVC 模式后，可以很方便的把要开发的系统切分成若干个部分，再在这若干个部分中再去切分，然后让不同的开发人员去开发。这样就非常有利于开发中的分工。比如，美工和网页设计人员进行视图部分的开发，Java 程序员进行模型和控制器部分的开发。

3. 有利于组件的重用[26]。MVC 模型把系统划分成了多个不同的模块，而这些模块就可以重复性的使用。比如，一些重复的商业逻辑可以单独做成一个模块，这样后期的开发中需要用到这个商业逻辑时就可以直接调用这个模块就可以了。界面上的显示通常需要重复使用，也可以做成单独的一个模块。

MVC 的缺点：

MVC 模式虽然有很多优点，但是也增加了系统开发的难度及复杂度。原来系统的开发是一体的，使用 MVC 模式要将系统的开发分成三个部分，三个部分的交互自然就会使开发的难度变大，而且代码量也可能会有所增加。但是综合前面的三个优势，MVC 模式的缺点是可以克服的。使用 MVC 以后，系统变得复杂了，对开发人员的技术水平要求比较高，同时模型和视图的分离也使得调试代码变得更困难一些[26]。瑕不掩瑜，尽管有不少的缺点，但是 MVC 仍然是构筑 Web 应用最好的架构模式[26]。

2.5 本章小结

本章着重讲述了全文检索的主要技术，详细介绍全文检索的基础概念，描述了 Lucene 的结构和特点，介绍了 MVC 设计模式。最后对系统开发所使用的 SSH 三个开源框架进行简单介绍。

第三章 费舍尔方法用于全文检索

本章介绍论文的核心算法，提出了一个能够对个人博客系统进行全文检索的算法，可以帮助用户实现多个关键词组的查询。本章主要内容为：朴素贝叶斯分类方法、费舍尔方法和算法流程及实现。

本论文起初打算使用朴素贝叶斯模型来完成全文检索，后因其在技术上难以实现，故使用了和朴素贝叶斯模型相近的费舍尔方法。因此，在本章开始处先简单介绍朴素贝叶斯模型，也为后面提出的费舍尔方法做一下铺垫。

3.1 朴素贝叶斯分类方法

3.1.1 贝叶斯定理

贝叶斯分类方法以贝叶斯定理为先决条件，采取了概率计算方法[24]。贝叶斯分类方法的本质就是通过计算新到达的样本在每个已有分类上的后验概率，然后把该新到达的样本判定为具有最大后验概率的分类[24]。在计算后验概率的过程中，需要计算数据集中每个分类的先验概率，及其属性的条件概率[24]。

3.1.2 概率论基础

1. 条件概率和乘法定理

在样本空间 S 中，设 A、B 是两个随机事件，在给定事件 A 发生的条件下，事件 B 发生的概率称为事件 B 在给定 A 的情况下的条件概率(也称后验概率)，记作 $P(B|A)$。对应地，$P(A)$ 则称为无条件概率(也称先验概率)。条件概率可以由下式计算：

$$P(B|A)=\frac{P(BA)}{P(A)} \quad \text{或者} \quad P(A|B)=\frac{P(A)}{P(B)}P(B|A). \tag{3-1}$$

概率的乘法定理可由条件概率得到：

$$P(AB)=P(B|A)P(A). \tag{3-2}$$

假设有 n 个事件 A_1，A_2，…，A_n，$n\geq 2$，则有：

$$P(A_1,A_2,...,A_n)=P(A_n|A_1A_2\cdots A_{n-1})P(A_{n-1}|A_1A_2\cdots A_{n-2})\cdots P(A_1). \tag{3-3}$$

2. 全概率公式和贝叶斯定理

在样本空间 S 中，A 为随机事件，B_1，B_2，…，B_n 为 S 的一个划分，且 $P(B_i)>0$(i=1，2，…，n)，则：

$$P(A)=P(A|B_1)P(B_1)+P(A|B_2)P(B_2)+\cdots+P(A|B_n)P(B_n)$$
$$=\sum_{i=1}^{n}P(A|B_i)P(B_i), \tag{3-4}$$

称上式为全概率公式。

由条件概率的定义和全概率公式可得贝叶斯定理：

$$P(B_j|A)=\frac{P(A|B_j)P(B_j)}{\sum_{i=1}^{n}P(A|B_i)P(B_i)}. \tag{3-5}$$

3. 事件的独立性

设 A， B 是两个随机事件，一般 A 和 B 发生的概率相互之间是有影响的，即 $P(B|A)\neq P(B)$，只有在这种影响不存在时才会有 $P(B|A)=P(B)$，这时有：

$$P(BA)=P(B|A)P(A)=P(A)P(B), \tag{3-6}$$

则称 A， B 为相互独立事件。

同样，对于 n 个事件 A_1， A_2，…， A_n，n≥2，如果有：

$$P(A_1A_2\cdots A_n)=P(A_1)P(A_2)\cdots P(A_n), \tag{3-7}$$

则称 A_1， A_2，…， A_n 为相互独立事件。

3.1.3 朴素贝叶斯分类器

朴素贝叶斯中的朴素指的是假设将要被组合的各个概率是彼此独立的[5][24]。即一个词组在属于某个指定分类的文档中出现的概率，与其他词组出现于该分类的概率是不相关的[24] [27]。构造朴素贝叶斯模型的最后一步是实际判定某个内容项所属的分类[6] [7] [9][24]。最简单的方法：计算被考查内容在每个不同分类中的概率，然后选择概率最大的分类[24]。该模型描述如图 3-1 所示：

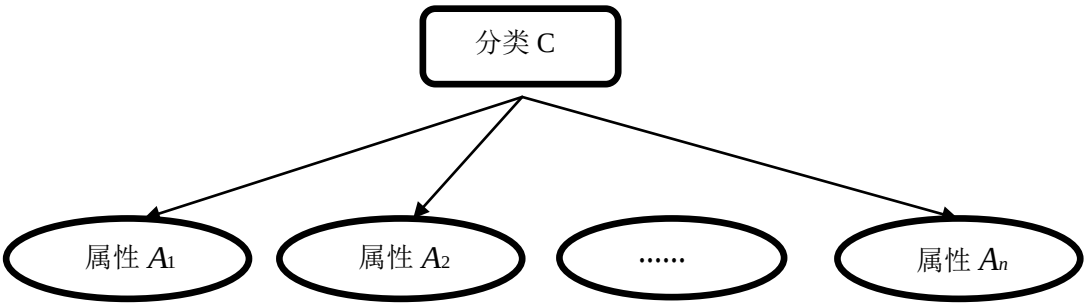


图 3-1 朴素贝叶斯模型

假设有一个变量集 U ={ A， C }，其中 A ={ A_1， A_2，…， A_n)包括 n 个条件属性。

$C=\{C_1, C_2, \cdots, C_m\}$，包含 m 个类标签。朴素贝叶斯分类模型假设所有的属性 A_i (i=1，2，…，n)都作为分类变量 C 的孩子节点。当且仅当 $P(C_i|X)>P(C_j|X)(1\le i,j\le m, j\ne i)$，将给定的一个待分类样本 $X=\{a_1, a_2, \cdots, a_n\}$分配给类 $C_i(1\le i\le m)$。

根据贝叶斯定理，有：

$$P(C_i|X)=\frac{P(C_i)P(X|C_i)}{P(X)}. \tag{3-8}$$

如果事先不清楚分类在数据集中的概率情况时，可以假设每个类别的概率相等[24]。即有：

$$P(C_i)=P(C_j)\ (C_i, C_j\in C \quad i\ne j\ , \tag{3-9}$$

并根据这个对 $P(C_i|X)$最大化。否则，最大化 $P(X|C_i)P(C_i)$。因 $P(X)$对于所有的类别均为常数，故有：

$$P(C_i|X)=\frac{P(C_i)P(X|C_i)}{P(X)}\propto P(C_i)P(X|C_i). \tag{3-10}$$

由朴素贝叶斯分类算法的条件属性相互独立的假设，有：

$$P(C_i|X)\propto P(C_i)\prod_{k=1}^{n}P(a_k|C_i), \tag{3-11}$$

其中 $P(C_i)=\frac{S_i}{S}$，S_i 是分类 C_i 在训练样本中的实例数，S 是训练样本总数。则 NBC 模型的公式表达式为：

$$NB(X)=\arg\max_{C_i\in C} P(C_i)\prod_{k=1}^{n}P(a_k|C_i), \tag{3-12}$$

概率 $P(a_1|C_i)$，$P(a_2|C_i)$，…，$P(a_n|C_i)$可由训练样本估值，其中：

1. 如果属性 A_k 是离散型的，则 $P(a_k|C_i)=\frac{S_{ik}}{S_i}$ 是在属性 A_k 上具有值 a_k 的分类 C_i 的训练样本个数。

2. 如果属性 A_k 是连续值，则一般假定它服从高斯分布[28]。因而，

$$P(a_k|C_i)=g(a_k, \mu_{C_i}, \sigma_{C_i})=\frac{1}{\sqrt{2\pi\sigma_{C_i}^2}}e^{\frac{(a_k-\mu_{C_i})^2}{2\sigma_{C_i}^2}}, \tag{3-13}$$

其中 $g(a_k, \mu_{C_i}, \sigma_{C_i})$ 是属性 A_k 的高斯密度函数，μ_{C_i}，σ_{C_i} 分别为平均值和标准方差。

3. 对于待分类样本 X，我们分别计算每个类别 $C_i\in C$ 的条件概率 $P(C_i)P(X|C_i)$。当且仅当 $P(C_i)P(C_i|X)>P(C_j)P(C_j|X)(1\le i,j\le m, j\ne i)$ 时，样本 X 属于类别 C_i。朴素贝叶斯分类模型的算法步骤：

（1）对所须要的数据集进行预处理，包括属性值离散化和缺失值填补；

（2）统计训练样本的个数 S，分类为 C_i 的样本数 S_i，分类 C_i 的样本中属性 A_k 取值为 a_k 的样本个数 S_{ik}；

（3）计算 $P(C_i)=\frac{S_i}{S}$ 和 $P(a_k|C_i)=\frac{S_{ik}}{S_i}$；

（4）利用分类模型 $NB(X)=\arg\max_{C_i\in C} P(C_i)\prod_{k=1}^{n}P(a_k|C_i)$ 得出待分类样本 X 的判定结果。

根据上述分析，朴素贝叶斯分类的流程可以由图 3-2 表示：

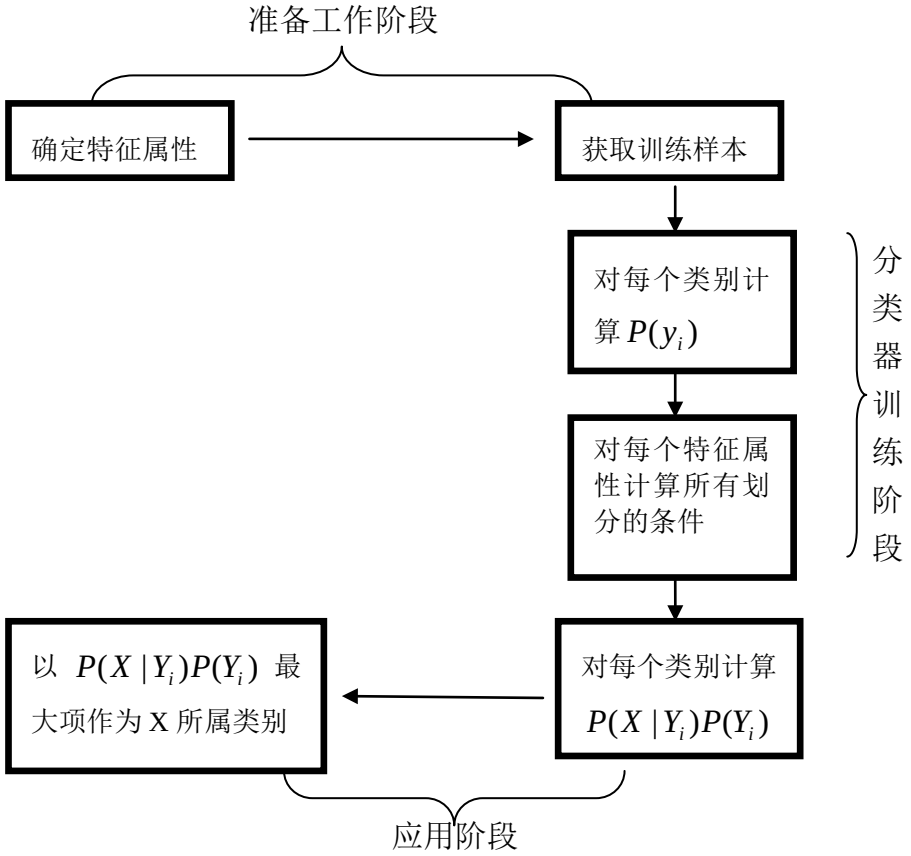


图 3-2 朴素贝叶斯分类的流程

朴素贝叶斯模型用于文档分类的算法流程如图 3-3 所示：

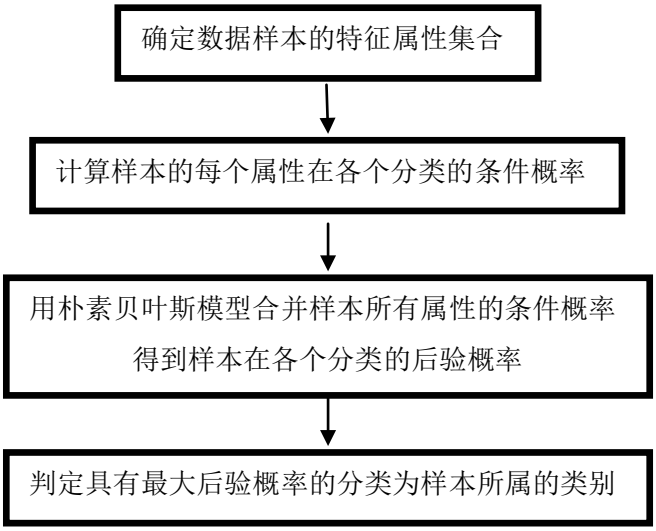


图 3-3 朴素贝叶斯模型用于文档分类

当使用朴素贝叶斯模型进行文档分类时，首先根据已有的文档及其分类构建朴素贝叶斯模型，然后对新到达的文档计算其出现在每个分类中的概率，最后取概率最大的分类并将其设定为新到达文档的分类，如图 3-4 所示：

图 3-4 朴素贝叶斯模型用于文档分类

从朴素贝叶斯模型用于文档分类的过程中，本论文提出了一个用于全文检索的算法。一篇博文由若干个分段构成。由已有的博文及其分段构成的朴素贝叶斯模型，对新到达的查询语句计算其出现在每一篇博文中的概率，排序后得到用户想要的查询结果。具体过程如图 3-5 所示：

图 3-5 朴素贝叶斯模型用于全文检索

当朴素贝叶斯模型用于文档分类时，最后一步将具有最大后验概率的分类判定给待分类的文档。在本论文提出的算法中，虽然仍然是构建朴素贝叶斯模型，但是最后一步是将所有博文的后验概率进行降序排序，然后将排好序的博文列表返回给用户。

从工程技术的角度来看，对一篇博文进行分段的话，编程实现上不是很精确。朴素贝叶斯模型通常用于垃圾邮件检测、文档分类等。在这些应用中，除了朴素贝叶斯模型外，还有一种其替代方案：费舍尔方法。在垃圾邮件检测的应用中，费舍尔方法直接计算后验概率，对每一个分类的阈值的设置更灵活，而且被人们认为比朴素贝叶斯模型检测垃圾邮件的效果更好。在应用到博文查询时，由费舍尔方法对应的方案将直接计算博文的分词，而无需再对博文进行分段，从而在技术上的编程实现比使用朴素贝叶斯模型更方便、更准确。费舍尔方法用于全文检索的算法流程如图 3-6 所示：

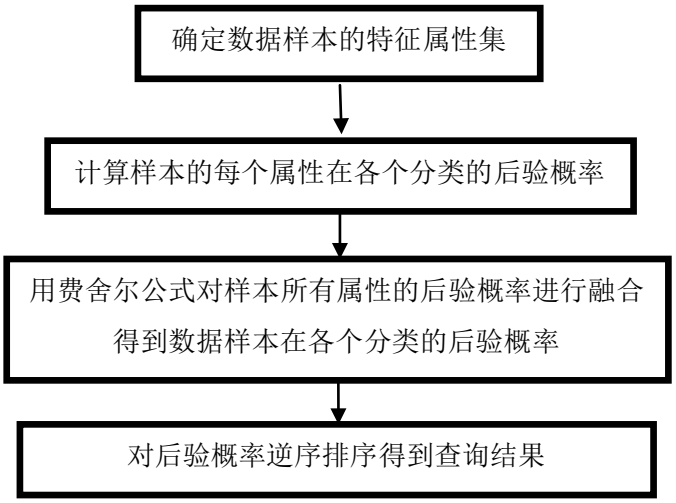


图 3-6 费舍尔方法用于全文检索

3.2 费舍尔方法

费舍尔方法由费舍尔创立。费舍尔全名 Ronald Aylmer Fisher，被人们认为是现代进化论的首席设计师之一，他创立了费舍尔方法以及雌雄双方的生物性状互相促进的进化理论，即“费舍尔氏失控理论”（Fisherian runaway），他是现代种群遗传学的的三杰之一(另外两杰是霍尔丹(J.B.S. Haldane)和莱特 (Sewall Wright))，是达尔文以来最伟大的生物进化学家。费舍尔对现代统计科学的重要贡献，包括方差分析，极大似然统计推断和许多抽样分布的导出。

在统计学里，费舍尔方法是一种数据融合的技术，用来合并在相同整体假设条件下的若干独立测试的结果。这里的 p_i 指的是第 i 次测试结果。

$$X_{2k}^2 \sim -2\sum_{i=1}^{k}\ln(p_i). \tag{3-14}$$

费舍尔方法用上面的公式把每次测试结果合并成卡方分布，最后用卡方分布的概率密度函数公式来计算最终的概率值。

费舍尔方法：若 n 个相互独立的随机变量 ξ_1、ξ_2、……、ξ_n ， 均服从标准正态分布，则

$$Q=\sum_{i=1}^{n}\xi_i^2 ,$$

构成的随机变量，其分布规律称为 χ_2 卡方分布。

卡方分布的基本描述：具有 k 个自由度的卡方分布是一个由 k 个独立标准正态随机变量的和所构成的分布。卡方分布经常用于我们常见的卡方检验中。卡方检验一方面可以用来衡量观测分布和理论分布之间的拟合程度，另一方面也可以测量定性数据两个分类标准间的独立性。事实上，卡方检验还有很多其它的作用。

卡方分布的定义：如果 Z_1， Z_2，.....， Z_k 是独立标准正态随机变量，那么这些变量的平方和就呈现出了 k 个自由度的卡方分布。平方和式子如下：

$$Q = \sum_{i=1}^{k} Z_i^2 . \tag{3-15}$$

通常，卡方分布可以表示为以下形式：

$$Q \sim \chi^2(k) \text{或者} Q \sim \chi_k^2 .$$

需要注意的是，卡方分布只有一个参数 k， k 是一个正整数，表明了分布中自由度的数目。

卡方分布的概率密度函数：

$$f(x,k) = \begin{cases} \dfrac{x^{(k/2-1)} e^{-x/2}}{2^{k/2}\Gamma(\frac{k}{2})}, & x > 0; \\ 0, & x \le 0. \end{cases} \tag{3-16}$$

$\Gamma(k/2)$ 表示的是一个伽玛函数，作为阶乘的延拓，伽玛函数 $\Gamma(x)$ 的定义为：

$$\Gamma(x) = \int_0^{+\infty} t^{x-1} e^{-t} dt . \tag{3-17}$$

3.3 算法流程及实现

本论文提出了一种全文检索的算法，该算法首先对输入的查询语句 query 进行中文分词并提取出若干查询词组，这些查询词组构成的向量就可以代表查询语句 query。然后计算每个查询词组在每一篇博文中出现的词频，并由该词频得到每个查询词组在每一篇博文中出现的后验概率。通过合并每篇博文中查询词组加权平滑后的后验概率得到查询语句出现在每一篇博文中的概率。在对此概率使用费舍尔方法后再对其进行排序输出即可得到用户想要的查询结果。在用户得到查询结果后，对用户选择的博文进行持续学习，从而不断改善已构建的模型。算法的主要流程如图 3-7 所示：

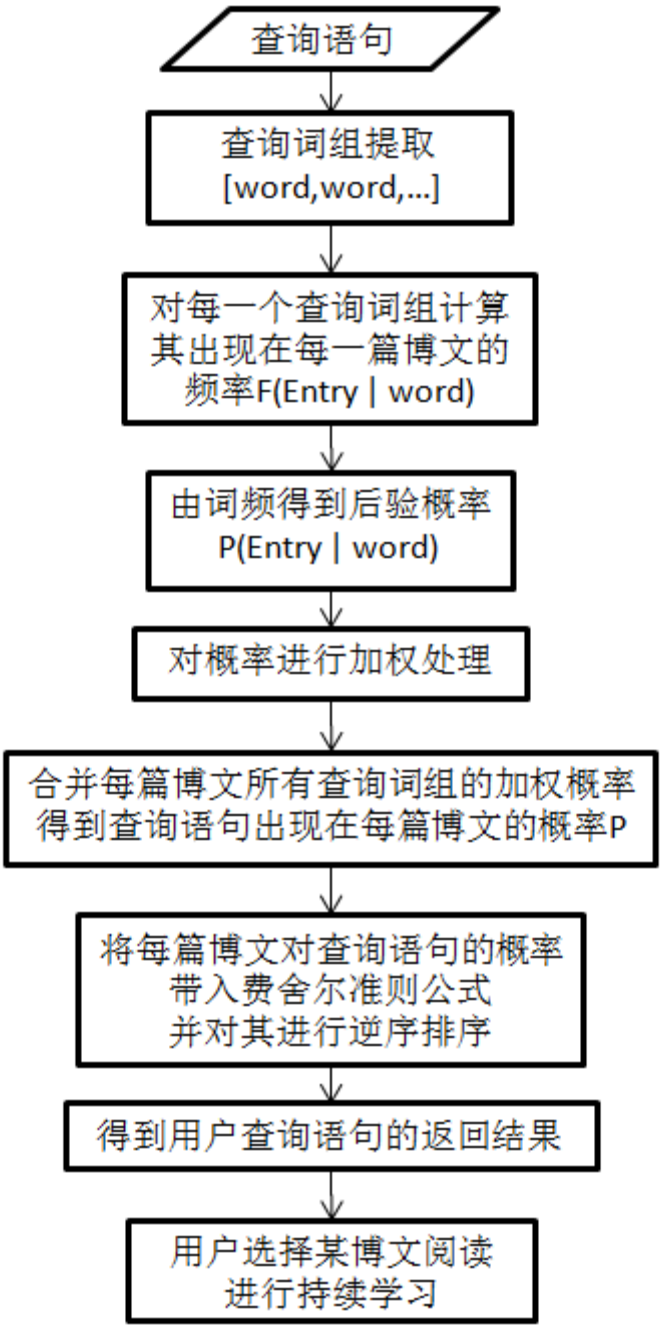


图 3-7 算法流程

由上图可知，算法一共有八个步骤，分别为：提取查询词组、计算查询词组的词频、计算查询词组的后验概率、对后验概率进行加权平滑处理、合并查询语句的后验概率、运用费舍尔方法、逆序排序概率得到查询结果和在点击中持续学习。此外，在算法开始之前，博主发表博文时需要对博文进行分词处理并将分词结果存储到数据库中。下面将对算法的各个步骤进行详细的介绍。

3.3.1 发表博文时进行预处理

在博主发表博文时，不仅要把博文完整的内容储存到数据库中，也要对博文进行中文分词，并把分词结果存到数据库中。如图 3-8 所示：

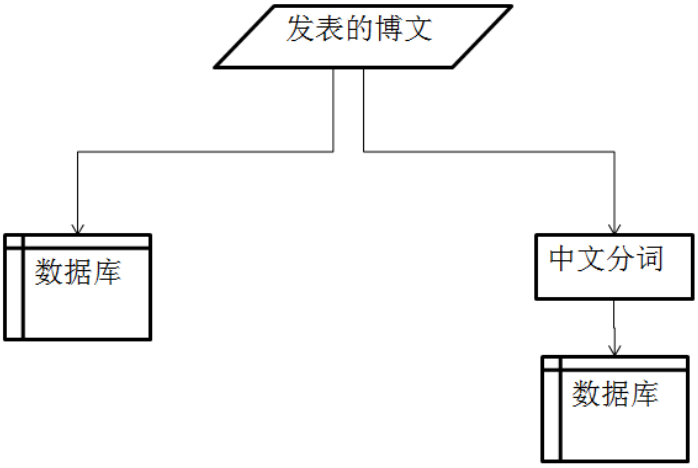


图 3-8 对发表的博文进行预处理

但是，仅仅对博文进行中文分词并不能得到很好的效果。现在的博客系统在发表博文时通常都使用所见即所得的组件。所见即所得的组件为博主撰写博文提供了便利，省去了博主自己编写 HTML 语言的博文内容。通过所见即所得的组件写出的博文虽然看起来简单明了，但是最终生成的仍旧是 HTML 语言标记的博文。这种博文中的 HTML 标记语言对查询没有任何帮助，反而降低了算法的执行效果，是算法输入数据中的“噪音”。为了不影响精确度，在执行算法前，一定要从博文数据中删除 HTML 标记语言，清除噪音。这就要求在中文分词之前删除 HTML 脚本。具体的实现方法就是删除博文中所有尖括号及其里面的 HTML 语言，例如<body>，</body>等。如图 3-9 所示：

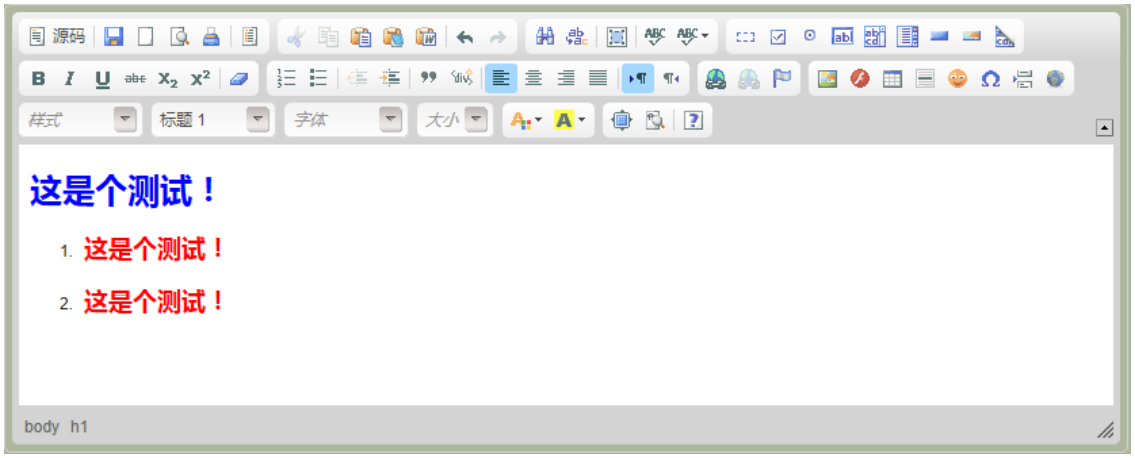


图 3-9 使用 CKEditor 组件撰写博文

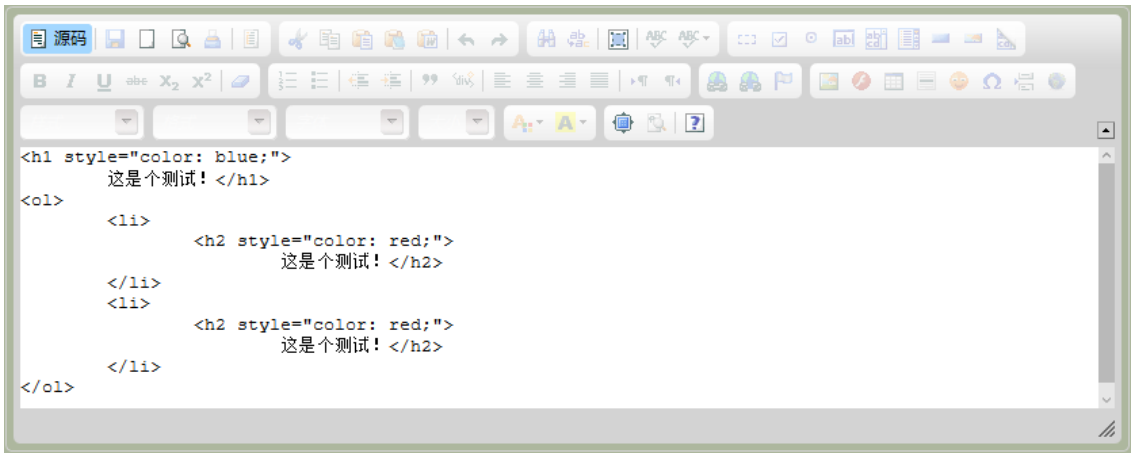


图 3-10 实际提交到数据库中的博文数据

3.3.2 提取查询词组

通过中文分词的方式对查询语句进行查询词组提取，得到查询语句的若干查询词组。这些查询词组可以看作成一个向量，而这个向量就可以代表查询语句。

$$query = [word_1, word_2, \ldots \quad , word_n]$$

3.3.3 计算查询词组的词频

计算每个查询词组在每篇博文中出现的次数，通过一定的方式得到每个查询词组在每篇博文中的词频。本系统只是简单的将每个查询词组出现在每篇博文中的次数除以该篇博文总的分词总数，从而得到该查询词组对该篇博文的词频。也可以使用更复杂的方法来计算词频以达到更好的运算结果，例如 lucene 中的词频计算方法。

$F(entry_1 \mid word_1)$	$F(entry_1 \mid word_2)$	...
$F(entry_2 \mid word_1)$	$F(entry_2 \mid word_2)$	...
$F(entry_3 \mid word_1)$	$F(entry_3 \mid word_2)$	...
...	...	...
...	...	...

3.3.4 计算查询词组的后验概率

计算每个查询词组出现在每篇博文的后验概率。

$$
\begin{array}{lll}
P_1(\text{entry}_1 \mid \text{word}_1) & P_1(\text{entry}_1 \mid \text{word}_2) & \ldots \\
P_1(\text{entry}_2 \mid \text{word}_1) & P_1(\text{entry}_2 \mid \text{word}_2) & \ldots \\
P_1(\text{entry}_3 \mid \text{word}_1) & P_1(\text{entry}_3 \mid \text{word}_2) & \ldots \\
\ldots & \ldots & \ldots \\
\ldots & \ldots & \ldots
\end{array}
$$

由上一步中计算得到的词频，通过下面的公式计算可得到后验概率，其中 k 为博文总数。

$$
P_1(\text{entry}_1 \mid \text{word}_1) = \frac{F(\text{entry}_1 \mid \text{word}_1)}{\sum_{j=1}^{k} F(\text{entry}_j \mid \text{word}_1)} \tag{3-18}
$$

这里的公式是本论文在参考应用于垃圾邮件检测中的费舍尔方法而仿照定制的，虽然逻辑上并不是很严谨，但是实际中是可以应用的。假设有分词总数相同的五篇博文，这五篇博文都含有分词 *word* 。通过上面的公式可以计算出他们的后验概率，从中可以发现，词频越大的博文经过计算后可以获得越大的后验概率。

在表 3-1 中，假设每篇博文的分词总数为 n。博文 $entry_2$ 中有 2 个分词 *word* ，则博文 $entry_2$ 对于分词 *word* 的后验概率为 0.2。计算过程为：

$$
P_1(entry_2 | word) = (2/n)/(0/n + 2/n + 5/n + 3/n + 0/n) = 0.2
$$

表 3-1 计算分词 word 的后验概率

分词总数相同的五篇博文	分词 *word* 的词频	计算后得到分词 *word* 的后验概率
$entry_1$	0/n	0.0
$entry_2$	2/n	0.2
$entry_3$	5/n	0.5
$entry_4$	3/n	0.3
$entry_5$	0/n	0.0

3.3.5 对后验概率进行加权平滑处理

使用下面的公式对上一步得到的后验概率进行加权平滑处理。

$$
\begin{array}{lll}
P_2(\text{entry}_1 \mid \text{word}_1) & P_2(\text{entry}_1 \mid \text{word}_2) & \ldots \\
P_2(\text{entry}_2 \mid \text{word}_1) & P_2(\text{entry}_2 \mid \text{word}_2) & \ldots \\
P_2(\text{entry}_3 \mid \text{word}_1) & P_2(\text{entry}_3 \mid \text{word}_2) & \ldots \\
\ldots & \ldots & \ldots \\
\ldots & \ldots & \ldots
\end{array}
$$

$$
P_2(\text{entry}_1 | \text{word}_1) = \frac{\text{weight} \times \text{assumeprob} + \text{count} \times P_1(\text{entry}_1 | \text{word}_1)}{\text{weight} + \text{count}} \tag{3-19}
$$

这里，assumeprob 是预先已知的后验概率，通常设定为一个合理的数值以达到平滑上一步计算出来的后验概率的目的； weight 为对 assumeprob 的权重，通常设置为 1；count 为查询词组在博文中出现的次数；$P_1(\text{entry}_1 \mid \text{word}_1)$为上一步计算出来的后验概率。

对上一步计算得到的概率进行加权平滑处理可以解决若干问题：

1. 解决零概率问题。所谓的零概率问题，就是在计算新查询语句的概率时，如果某个查询词组在训练集中从未出现过，会导致整个查询语句的概率计算结果为 0。针对本文中的算法就是当一个查询词组在训练集中没有出现过，那么该查询词组的后验概率为 0，在算法的后面步骤中使用连乘法计算查询语句的后验概率时，整个查询语句出现的概率也为 0。这显然是不合理的，因为不能因为一个事件没有观测到就判断该事件的概率为 0。

朴素贝叶斯模型的最后一步要使用连乘法计算最后的后验概率。当某个分词在博文中没有出现过，那么该分词的概率为 0，使用连乘法计算最后的概率时会导致整个结果为 0,这会影响精度而且也不合理。通过加权平滑处理可以方便有效的避免零概率问题。为了得到更好的效果，本算法使用了上式来取代常用的拉普拉斯算法。

2. 解决冷启动问题。仅仅依据以往见过的信息，会令算法构建的模型在训练的初期阶段，对那些极少出现的分词变得异常敏感。在模型训练的初期阶段，模型构建的尚不完善，这样对那些从未出现的词组会很极端。通过使用上面的公式对后验概率进行加权处理，可以有效地缓和冷启动问题。

3. 使用已有的数据。有可能我们已经掌握了更多的背景信息，从而使用已有的数据来完善系统。通过使用上面的公式也可以使用之前已经有的数据，从一个合理的推测开始。

例如，假设通过已有的数据得知 $word_1$ 在 $entry_1$ 中的后验概率为 0.1，则在后续的查询中可以直接使用已有的数据。方法是设置上面公式中的 assumeprob 为 0.1。

3.3.6 合并查询语句的后验概率

合并查询词组的加权概率，得到查询语句关于每篇博文的概率。

$$
\begin{gathered}
P_3(\text{entry}_1 \mid \text{query}) \\
P_3(\text{entry}_2 \mid \text{query}) \\
P_3(\text{entry}_3 \mid \text{query}) \\
\cdots \\
\cdots
\end{gathered}
$$

$$P_3(entry_1 | query) = \prod_j P_2(entry_1 | word_j) \tag{3-20}$$

朴素贝叶斯模型的最后一步就是将各个属性的后验概率根据相互独立事件概率的乘法原则进行连乘以计算最后的后验概率。在本论文提出的算法中，通过合并一篇博文中所有查询词组的概率，得到该篇博文对整个查询语句的后验概率。

3.3.7 运用费舍尔方法

在上一步须要将对应各个查询词组的后验概率值组合起来,形成一个总的关于查询语句的概率值。理论上，可以将各个查询词组的概率值连乘起来，利用相乘的结果在不同博文间进行比较。当然，由于查询词组不是彼此独立的，因此计算出来的结果并不代表真实的概率。由费舍尔方法返回的结果是对概率的一种更好的估计。

费舍尔方法将查询语句的加权概率带入费舍尔方法，得到查询语句关于每篇博文的概率。费舍尔方法把独立同分布的若干事件融合成卡方分布，再根据卡方分布的概率密度函数计算最后的概率。

$$\mathrm{P}(\mathrm{entry}_1 \mid \mathrm{query})$$
$$\mathrm{P}(\mathrm{entry}_2 \mid \mathrm{query})$$
$$\mathrm{P}(\mathrm{entry}_3 \mid \mathrm{query})$$
$$\cdots$$
$$\cdots$$

$$\mathrm{P}(\mathrm{entry}_1 \mid \mathrm{query}) = \sum_{i=0} \frac{\left(\frac{p}{2}\right)^i}{i!} e^{-\frac{p}{2}}, p = -2\ln\left(\mathrm{P}_{\mathrm{i}}\left(\mathrm{entry}_1 \mid \mathrm{query}\right)\right) \tag{3-21}$$

费舍尔方法的计算过程是将所有的概率相乘起来，然后取自然对数，再将所得结果乘以-2。费舍尔方法告诉我们，如果概率彼此独立且随机分布，则这一计算结果将满足对数卡方分布。在实际应用中，不属于某个博文的查询语句中，可能会包含针对该博文的不同属性概率的查询词组（可能会随机出现）；或者，一个属于该博文的查询语句中会包含许多概率值很高的属性。通过将费舍尔方法的计算结果传给倒置对数卡方函数，我们会得到一组随机概率中的最大值。

3.3.8 逆序排序概率得到查询结果

对查询语句出现在每篇博文的概率进行逆序排序，得到查询结果。

$$\mathrm{P}(\mathrm{entry}_2 \mid \mathrm{query})$$
$$\mathrm{P}(\mathrm{entry}_1 \mid \mathrm{query})$$
$$\mathrm{P}(\mathrm{entry}_3 \mid \mathrm{query})$$
$$\cdots$$
$$\cdots$$

$$\mathrm{P}(\mathrm{entry}_2 \mid \mathrm{query}) > \mathrm{P}(\mathrm{entry}_1 \mid \mathrm{query}) > \mathrm{P}(\mathrm{entry}_3 \mid \mathrm{query}) \ \ldots$$

为了得到更好的用户体验，使系统更人性化，这里可以设置一阈值λ，对小于此阈值的博文不再呈现给用户。当有不符合用户查询要求的博文时，给用户返回不包含这些博文的查询结果更人性化、更智能化。例如，假设$entry_3$博文小于设定的阈值，则只返

回前面两篇的博文 $entry_2$ $entry_1$ 会更符合用户的查询要求。

3.3.9 在点击中持续学习

用户输入查询语句后，会得到查询的返回结果，之后用户选择某篇博文进行阅读。此时，不仅需要把这篇博文从数据库中取出来呈现给用户，还应进行后续的学习，对用户选择的博文进行持续学习。为了改善用户的使用体验，完善算法构建的模型，可以继续对模型进行持续学习。即，对用户选择的博文增加从用户查询语句提取的查询词组，强化该篇博文对查询词组的学习，从而提高在给定的查询语句下该篇博文最后计算出来的概率。通过多次学习后，该篇博文会在同样的查询语句下的返回结果中得到越来越高的排序名次。如图 3-11 所示：

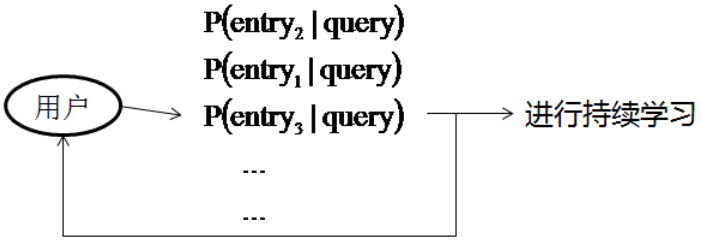


图 3-11 在点击中持续学习

假设现在有查询语句 $query$，经过分词后得到查询词组 $word_1$ 和 $word_3$。用户在得到查询结果后选择了博文 $entry_3$ 来进行进一步的阅读。这时要对算法构建的模型进行持续学习，即使博文 $entry_3$ 的分词列表中对应于查询词组 $word_1$、$word_2$ 分别加 1。之后，对于同样的查询语句 $query$， $entry_3$ 计算的最后概率将会有所增大。如表 3-2 所示：

表 3-2 对用户选择的博文进行持续学习

博文 $entry_3$ 的所有分词的列表	$word_1$	$word_2$	$word_3$	.....
未进行学习前 $entry_3$ 的博文分词	n_1	n_2	n_3	.....
用户选择 $entry_3$ 进行阅读， 查询语句为 $query=[word_1\ \ word_3]$				
则持续学习后 $entry_3$ 的博文分词	n_1+1	n_2	n_3+1	.....

3.3.10 进一步的改进

1. 在整个算法的过程中，可以使用全文检索常用的开源组件 lucene 的词频。lucene 的词频设计的很完善，可以调用 lucene 接口并得到 lucene 组件计算后得到的词频并直接代入本文算法的第三步中的 F(entry|word)。如图 3-13 所示：

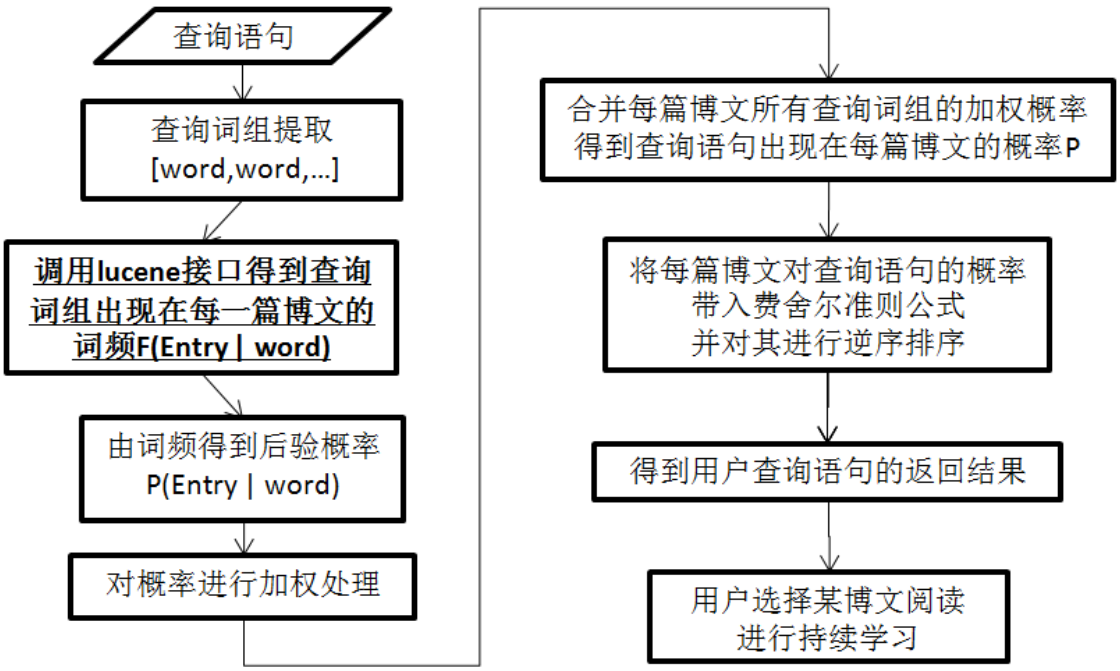


图 3-13 算法流程

2. 用户查询的结果，不仅仅只注重词频，同时也关注其他的参数。比如说博文的浏览次数，博文的评论数，博文的发表时间等。对于内容相似的博文，近期发表的博文通常比很早之前的博文更符合用户的查询请求。这样可以在计算出最终的概率后加入其他参数，并对此参数设置一个权重值，再对最后计算出来的加权概率进行排序。这样的查询结果可能更符合用户的要求。如图 3-14 所示：

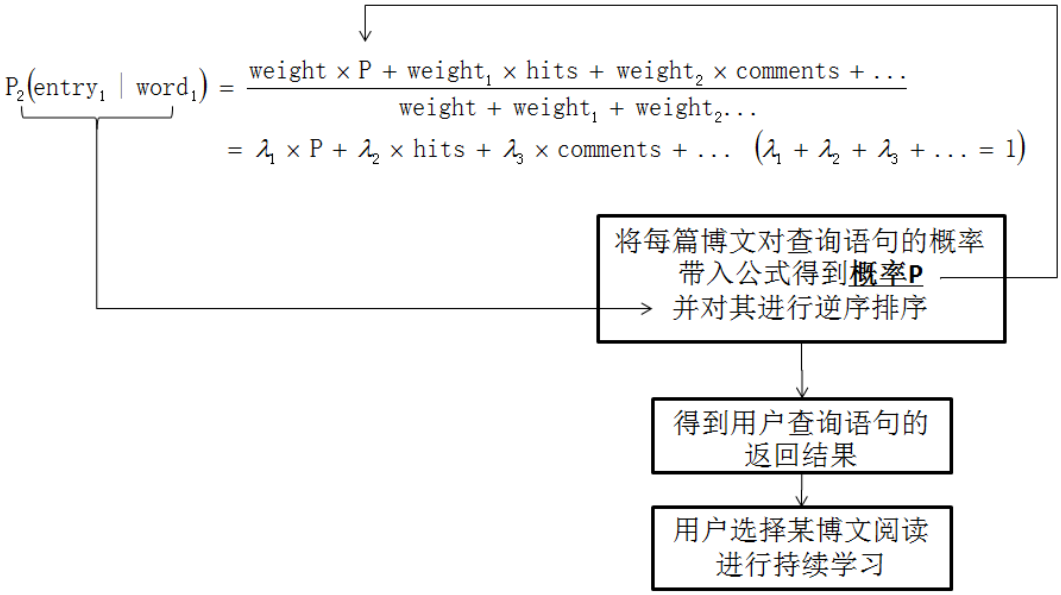


图 3-14 对最后的后验概率加入其它参数

3.3.11 举例说明

在本节，本论文将通过举例来演示上述算法的计算过程。

假设目前有五篇博文：

1．永远对陌生人保持警惕，尤其是主动帮忙并显得过分热情的人。对于自己不熟悉的事情，密切关注助人者的一举一动，避免自己受伤害。------刘墉

2．一个志在有大成就的人，它必须如歌德所说，知道限制自己。反之，什么事都想做的人，其实什么事都不能做，而终归于失败。------黑格尔

3．人应尊敬他自己，并应自视能配得上最高尚的东西。------黑格尔

4．我首先要求诸君信任科学，相信理性，信任自己，并相信自己。------黑格尔

5．自由是对必然的认识 。无知者是最不自由的，因为他要面对的是一个完全黑暗的世界。------黑格尔

当将这五篇博文存储到数据库中时，实际存储到数据库中的博文数据包含大量的 HTML 标记脚本。在对博文进行分词前要删除博文中的 HTML 脚本。如图 3-15 所示：

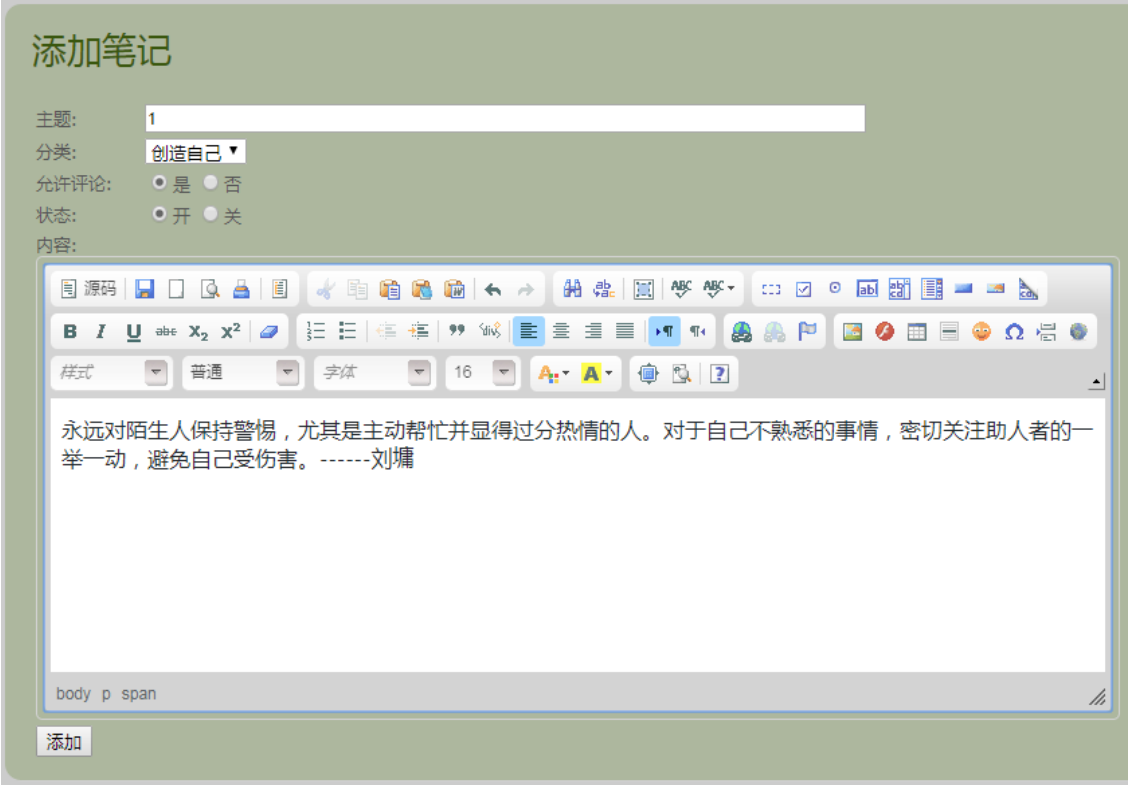


图 3-15 博文 1 的内容

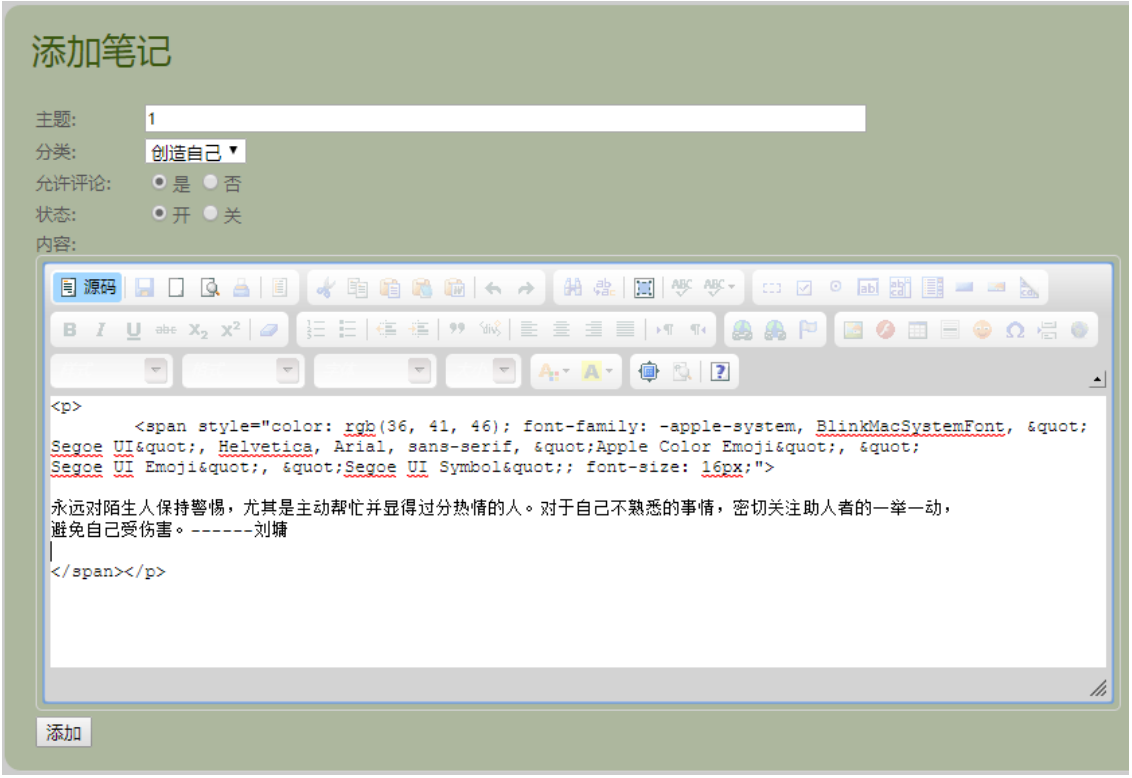
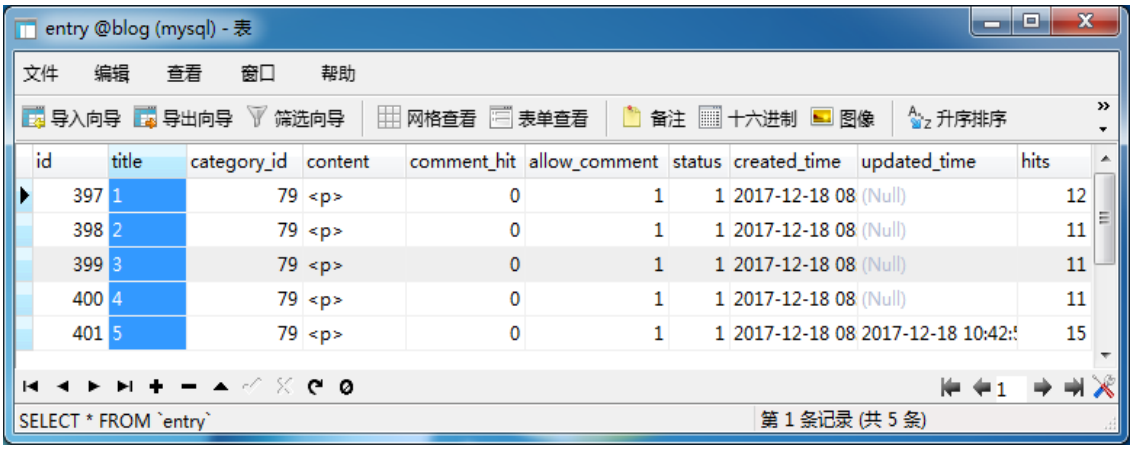


图 3-16 博文 1 提交到数据库中的内容

存储到数据库后，数据库里的博文数据如图 3-17 所示：



id	title	category_id	content	comment_hit	allow_comment	status	created_time	updated_time	hits
397	1	79	<p>	0	1	1	2017-12-18 08	(Null)	12
398	2	79	<p>	0	1	1	2017-12-18 08	(Null)	11
399	3	79	<p>	0	1	1	2017-12-18 08	(Null)	11
400	4	79	<p>	0	1	1	2017-12-18 08	(Null)	11
401	5	79	<p>	0	1	1	2017-12-18 08	2017-12-18 10:42:	15

图 3-17 五篇博文存储在数据库中的内容

按照上述算法的步骤，下面依次进行说明：

首先将博文和对博文分词后的结果存储到数据库里，分词后的结果如下所示：

1．永远/对/陌生/生人/人保/保持/警惕/尤其/尤其是/主动/帮忙/显得/得过/过分/热情/的人/对于/自己/不熟/熟悉/熟悉的/的事/事情/密切/关注/助人/者/一举/一动/避免/自己/受伤/伤害/刘墉/

2．一个/志在/在有/大/大成/成就/的人/必须/歌德/所说/知道/限制/自己/什么事/都想/想做/做的/的人/其实/什么事/都不/不能/能做/终归/归于/失败/黑格尔/

3．应/尊敬/自己/他自己/应/自视/能/配得/得上/最高/高尚/高尚的/东西/黑格尔/nbsp/

4．首先/先要/要求/诸君/信任/科学/相信/理性/信任/自己/相信/自己/信自己/黑格尔/

5．自由/由是/是对/必然/认识/无知/知者/是最/最不/不自/自由/因为/为他/他要/面对/

对的/的是/一个/完全/黑暗/暗的/世界/黑格尔/

当查询语句为“相信”，使用庖丁解牛分词器进行分词时仍旧是“相信”。按照本节介绍的算法计算结果如下所示：

表 3-3 查询语句为“相信”的算法计算结果

id	397	398	399	400	401
title	1	2	3	4	5
词频	0.0	0.0	0.0	0.142857143	0.0
后验概率	0.0	0.0	0.0	1.0	0.0
加权平滑	0.016666666	0.016666666	0.016666666	0.683333333	0.016666666
费舍尔方法	0.016666667	0.016666667	0.016666667	0.683333333	0.016666667

对根据费舍尔方法计算出的最后的博文概率进行逆序排序可得：

4 > 2 > 1 > 3 > 5

系统的运行结果如图 3-18 所示：

图 3-18 查询语句“相信”的查询结果

当查询语句为“自己”，使用庖丁解牛分词器进行分词时仍旧是“自己”。按照本节介绍的算法计算结果如下所示：

表 3-4 查询语句为“自己”的算法计算结果

id	397	398	399	400	401
title	1	2	3	4	5
词频	0.058823529	0.037037037	0.066666667	0.142857143	0.0
后验概率	0.192621280	0.121280065	0.218304117	0.467794537	0.0
加权平滑	0.172246811	0.111097199	0.194260672	0.408109603	0.007142857
费舍尔方法	0.172246811	0.111097199	0.194260672	0.408109603	0.007142857

对根据费舍尔方法计算出的最后的博文概率进行逆序排序可得：

4 > 3 > 1 > 2 > 5

系统的运行结果如图 3-19 所示：

图 3-19 查询语句“自己”的查询结果

通过对查询语句“相信”、“自己”计算结果可知，加权平滑后的概率与用费舍尔方法计算后的概率相同。这是由于对查询语句“相信”、“自己”分词后的结果只包含一个查询词组，而费舍尔方法是一种数据融合技术。当只有一个事件时，费舍尔方法没有多余的数据来进行融合，从而导致对于查询语句只有一个查询分词时，使用费舍尔方法后并不能改变已经加权平滑的概率。这也说明了本论文实现的个人博客系统的全文检索在算法逻辑上是正确的，即正确地编程实现了本论文提出的算法。

当查询语句为“信任自己”时，使用庖丁解牛分词器进行分词，分词结果为“信任”、“自己”两个查询分词。按照本节介绍的算法计算结果如下所示：

1. 计算分词“信任”的词频

表 3-5 分词“信任”的词频

id	397	398	399	400	401
title	1	2	3	4	5
词频	0.0	0.0	0.0	0.142857143	0.0

这里以博文 4 为例，博文 4 的分词结果为：

首先/先要/要求/诸君/信任/科学/相信/理性/信任/自己/相信/自己/信自己/黑格尔/

在博文 4 的分词结果中，一共有 14 个分词，其中有两个“信任”分词。对博文 4 计算“信任”的词频为 2/14=0.142857143。

2. 计算分词“信任”的后验概率

表 3-6 分词“信任”的后验概率

id	397	398	399	400	401
title	1	2	3	4	5
后验概率	0.0	0.0	0.0	1.0	0.0

对博文 4 计算分词“信任”的后验概率为

0.142857143/（0.0+0.0+0.0+0.142857143+0.0）=1.0

3. 对分词“信任”的后验概率进行加权平滑处理

这里使用 3.3.5 节中的公式来对后验概率进行加权平滑处理。本论文在本例中设置 weight 为 1，设置 assumeprob 为 0.05，count 为查询词组“信任”在博文 4 中出现的次数，P_1为上一步计算出来的后验概率。

$$P_2 = \frac{\text{weight} \times \text{assumeprob} + \text{count} \times P_1}{\text{weight} + \text{count}} = \frac{1 \times 0.05 + 2 \times 1.0}{1+2} = 0.683333333$$

表 3-7 分词“信任”加权后的概率

id	397	398	399	400	401
title	1	2	3	4	5
加权平滑	0.016666666	0.016666666	0.016666666	0.683333333	0.016666666

按照 1 到 4 步骤对查询词组“自己”计算结果如下

表 3-8 分词“自己”加权后的概率

id	397	398	399	400	401
title	1	2	3	4	5
词频	0.058823529	0.037037037	0.066666667	0.142857143	0.0
后验概率	0.192621280	0.121280065	0.218304117	0.467794537	0.0
加权平滑	0.172246811	0.111097199	0.194260672	0.408109603	0.007142857

4. 对查询分词的后验概率使用费舍尔方法，合并“信任”、“自己”两个查询词组的最后计算出的概率，得到查询语句的最后计算出来的概率。当查询语句为“信任自己”时，计算博文 4 关于查询语句“信任自己”的使用费舍尔方法后的概率时，首先是计算 $P = -2(\ln(P(entry_4 | word_1)) + \ln(P(entry_4 | word_2)))$

$P = -2(\ln(0.683333333) + \ln(0.408109603)) = 2.553984004$；然后将 P 带入卡方分布的概率密度公式，计算后可得博文 4 关于查询语句“信任自己”的最后概率：$P_0 = e^{-\frac{P}{2}} + \frac{p}{2} \times e^{-\frac{P}{2}} = (1 + 2.553984004/2) \times e^{-2.553984004/2} = 0.634995907$。

表 3-9 用费舍尔方法合并查询分词的后验概率

id	397	398	399	400	401
title	1	2	3	4	5
费舍尔合并	0.019673949	0.013501447	0.021798958	0.634995907	0.001194760

5. 对查询语句计算出的最后概率进行逆序排序，得到查询语句的查询结果。对根据费舍尔方法计算出的最后的博文概率进行逆序排序可得：

4 > 3 > 1 > 2 > 5

系统的运行结果如图 3-20 所示：

图 3-20 查询语句“信任自己”的查询结果

3.4 本章小结

本章提出了一个可以用于全文检索的算法，并具体介绍了算法每一步的详细过程。本章所论述的主要是该算法的理论基础和原理讲解。

在下一章节中将介绍系统的设计与实现，并着重介绍如何在 SSH 框架上实现两种全文检索方案：用费舍尔方法实现的全文检索和用 lucene 组件实现的全文检索。

第四章 系统的设计与实现

本章介绍如何用 SSH 框架来编程实现博客系统，并在此博客系统上实现了两种全文检索的方案。本章主要内容为：SSH 框架构建博客系统、费舍尔方法实现全文检索、lucene 实现全文检索及两种方案的对比。

4.1 SSH框架构建博客系统

由于Java语言具有跨平台等特点[25]，因此Java语言常被用来进行Web系统的开发。本系统选择 Java 语言作为编程语言，使用 SSH 框架开发博客系统，系统的开发可以分为三个部分：系统需求分析，系统设计和编程实现。

4.1.1 系统需求分析

一个系统的开发与实现，首先要对其进行设计分析，确定系统需要实现哪些功能，注意什么问题。系统的需求分析就是将系统要实现的功能抽象出来，然后进行详细设计，最后将其实现。具体要求如下：系统需求分析的核心问题是考虑当前系统会涉及到哪些用户以及这些用户需要使用什么样的功能：

1. 确定用户。分析之后可以得出，博客系统只涉及到两种用户：博客的主人，简称为博主；浏览博客内容、评论博客的人，简称为浏览者。

2. 系统的功能。博主使用的功能：系统管理、文章管理、分类管理和友情链接管理等；浏览者的功能： 浏览文章、评论文章和订阅博客等。

本系统使用 starUML 来设计博客系统的用例图。用例图的设计也可以使用 IBM Rational Rose 来设计。博客系统用例图如图 4-1 和图 4-2 所示：

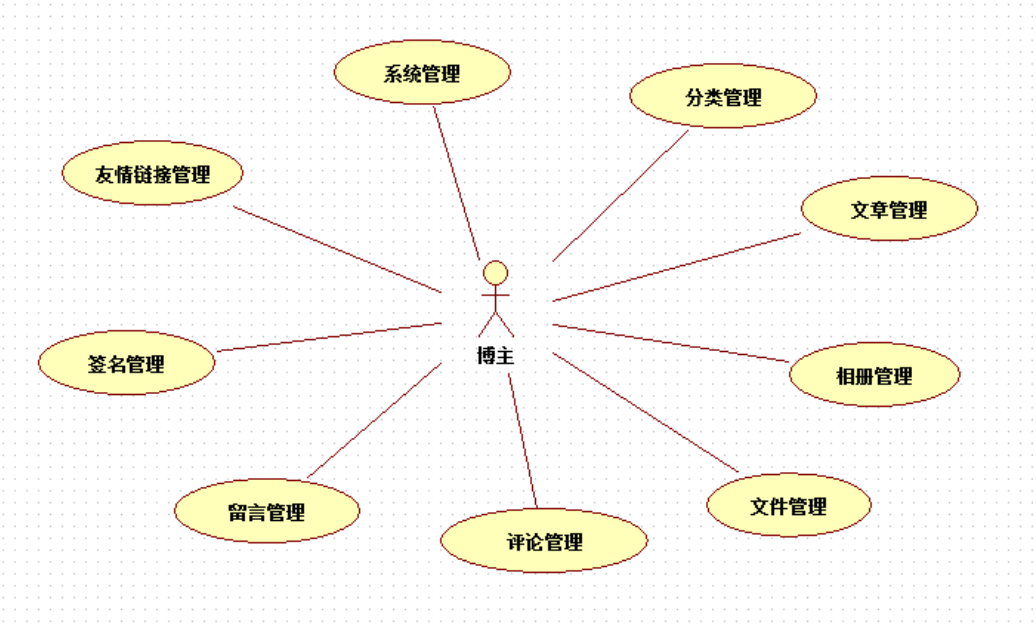


图 4-1 博主用例图

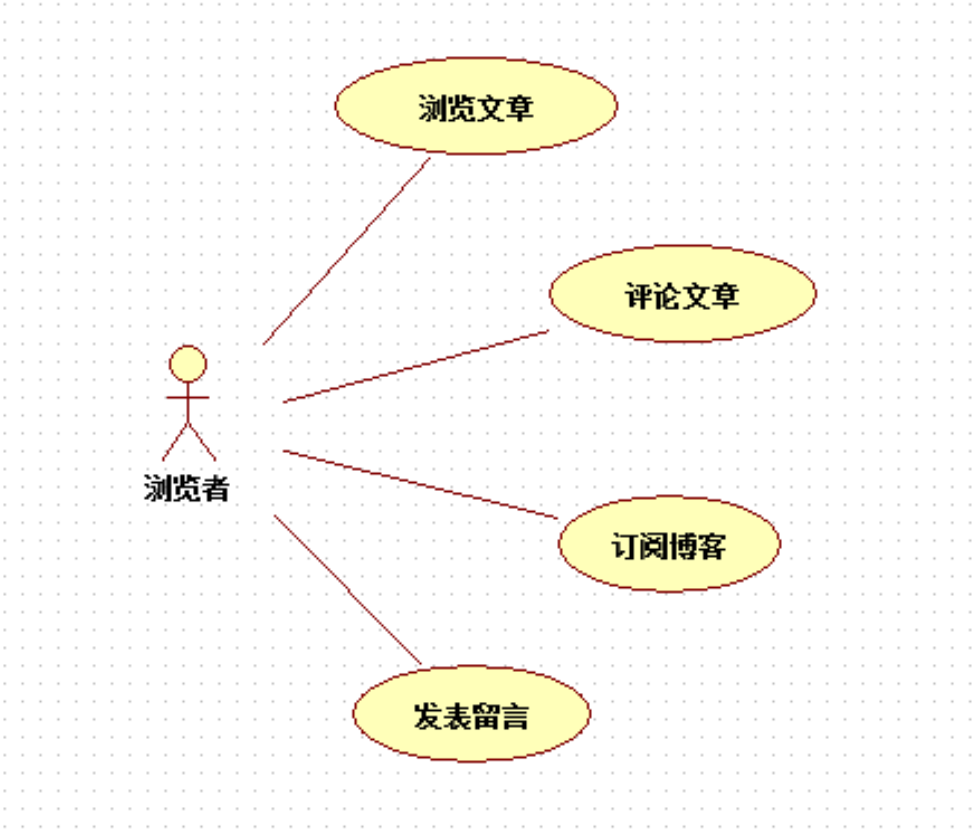


图 4-2 浏览者用例图

4.1.2 系统设计

1. 系统架构设计

Client/Server 的四层架构如图 4-3 所示。分层设计不仅增强复用性，把功能不同的代码放在不同的层次上，而且使结构上更加清晰，以后出了问题也可以修改这一层的代码而不需要从整体代码上修改。

数据层主要完成对这个系统相关数据的存储。这里自然指的是使用的数据库。本系统使用 MySQL 作为数据库。

持久层主要完成系统中数据的持久化，通常使用一些持久层框架来便利系统的开发。本系统使用的是 Hibernate。

业务逻辑层又称服务层，主要完成这个系统业务逻辑的运算。

表现层主要用来实现用户的交互。

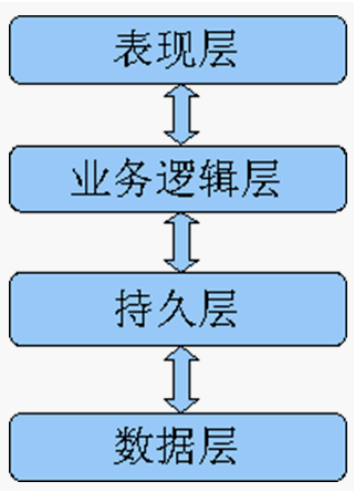


图 4-3 C/S 四层架构

开发的模式一般有两种，自顶层往底层设计，也可以从底层向顶层开始层层设计。本系统的开发采用后一种方法。

2. 数据库设计

本系统使用 PowerDesigner 来进行数据库设计。 当使用 PowerDesigner 去设计系统中的数据库时，一般会先设计出概念模型，物理模型，最后再根据物理模型生成数据库所对应的建表语句，这样就可以完成整个系统的数据库设计。

概念模型的设计首先要找到当前系统中要用到的实体，也就是对应要用到的表；同时找到这些实体当中有哪些属性值需要保存，也就是表中需要哪些字段；最后找到实体与实体之间有哪些是一对一的关联，哪些是一对多的关联。

物理模型的设计要使用 Powerdisigner 由概念模型来生成物理模型。这样生成的物理模型可能有一些小的问题，检查改正之后，即可使用 Powerdesigner 由物理模型生成数据库建表语句。

最后使用 Navicate 来管理操纵数据库。首先使用 Navicate 连接 MySQL 数据库，新建数据库连接 mysql，之后新建数据库 blog，进入到 blog 数据库后执行由 PowerDesigner 生成的建表语句即可完成数据库的设计。

数据库的整体结构如图 4-4 所示：

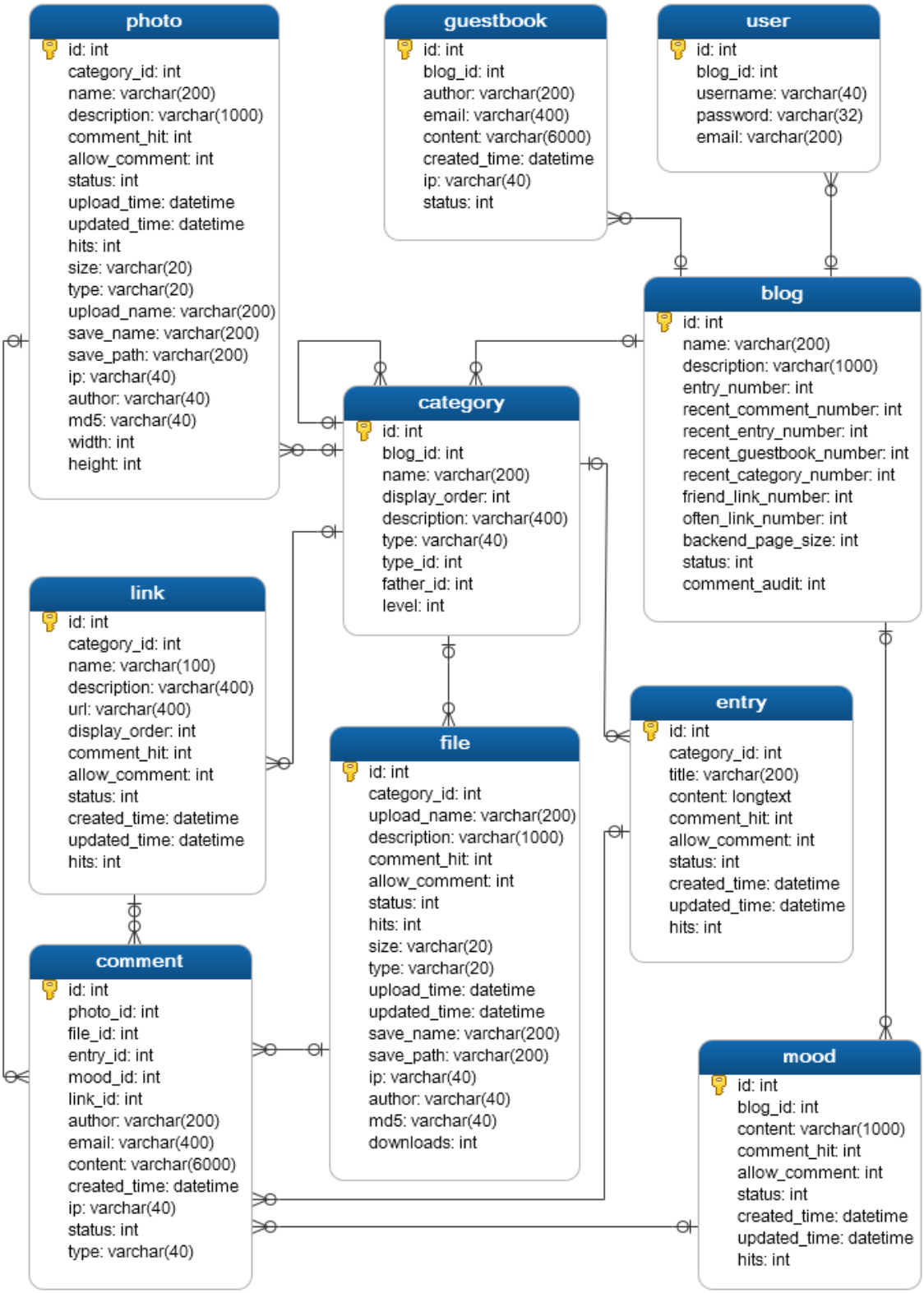


图 4-4 系统数据库的整体结构

特别需要注意的是，User 表中的 password 字段保存的是当前博主的用户密码，这个密码在数据库中并不是明文保存的，而是保存经过单向加密以后的字符串值，所以数据库在此字段上设置了 32 位。

3. 域模型的设计

本系统使用 Hibenate 来进行域模型设计。域模型的设计实际上也就是对持久化类的设计。这里可以在集成开发环境 MyEclipse 下使用 Hibernate 逆向工程由数据库里的表直接逆向生成与表相对应的映射对象类文件。比如，由数据库中的 entry 表可以逆向映射生成一个Entry类，之后就可以直接用面向对象的方法来对数据库中的entry表进行操作。一个 entry 对象就表示存储在表 entry 中的一篇博文的信息。

4. 持久层设计

持久层设计采用泛型的 DAO 设计。DAO（Data Access Object，数据访问接口）设计模式是 J2EE（Java2 平台企业版）的核心模式。这种设计模式很好的将底层数据访问与业务逻辑实现高度分离，这样便于开发人员专注于开发所需的功能实现代码，而不需要过多地考虑上下层交互的问题，很好地降低了出错的几率，提高了开发效率。

5. 业务逻辑层的设计

当在 DAO 中设置事务控制时，如果一个事务需要跨越多个 DAO 实现类的不同方法时，则无法直接使用 Spring 的注解设置事务的边界。在没有业务逻辑层的情况下，有些事务是无法进行设置的。

例如，在当前博客系统中，当博主删除一篇文章时，应该同时删除这篇文章所对应的评论。这是两个步骤，要么都成功，要么都回滚，应该放在一个事务当中。但是这两步分别由两个不同的 DAO 对象来完成操作，一个是 entryDao 来完成的操作，一个是 commentDao 来完成的操作。现在就跨越了两个 DAO，从而无法完成操作。为了更好设计事务，以及更高的将来代码复用，这样就需要增加一个层次，让这个层次上的代码得到一些复用。从而更好的去控制事务的边界。这里就需要提供业务逻辑层，又叫服务层。在业务逻辑层当中去设计事务的边界，这样的话，对底层的操作，即使跨越了多个DAO，不管是几个，都可以在业务逻辑层正确的设置事务。因为一个业务逻辑层当中的一个方法可能使用了多个 DAO，这样就可以在这个方法上去设置事务的处理。这样就把多个 DAO 的操作放在了一个事务当中。如图 4-5 所示：

```
//entryDao与commentDao是两个不同的Dao对象，
//但是这两步操作应该放到一个事务中进行控制
entryDao.delete(entry);
commentDao.deleteByEntryId(id);
```

图 4-5 删除博文的同时删除该博文下的评论

6. 表现层的设计 本系统使用 Struts2 来实现表现层的功能。

7. 系统详细设计

对系统进行详细设计主要是对 Action 类的设计。 每个 Action 类实现一个对应的功能。例如，EntryAction 实现关于博文的相关功能，包括博客文章的分页列表显示、添加、修改和删除等功能。

8. 权限控制的设计

对权限控制的设计主要是对博主和浏览者的权限的设计。本系统使用 Servlet 过滤

器来实现权限控制。为了配合 Servlet 过滤器的使用，系统使用路径去检查权限。即把博主才能访问的资源和页面都保存在当前博客系统的“/admin”目录下，同时涉及到的 Action 也映射到“/admin”这个命名空间下。

9. 分页设计

不管博客的前台还是后台都需要分页显示，本系统专门设计了 Pagination 类来保存分页的信息。Pagination 类包括分页的大小，分页的内容，当前的页码，总共的记录数等分页内容。此外，本系统在 JSP 页面上使用 Pager 标签库的标签来实现分页链接的显示。

10. 数据校验的设计

在博客系统当中，有很多地方需要数据校验。例如，添加文章、用户登录和用户添加评论的时候，有些项不能为空，也不能不符合规则。为了简化这个工作，这里使用的是 Struts2 数据校验框架。使用 Struts2 的校验框架只需要去编写对应的 XML 映射配置文件即可对前台输入数据的校验，既可以是服务器端的校验，也可以是客户端的校验。

11. 开发环境的设定

本系统采用 Java 语言开发，JDK 版本为 JDK1.6。

操作系统：Microsoft Windows XP

OO 设计：starUML

数据库设计：Sybase PowerDesigner

数据库系统：MySQL5.0

相关工具包：Lucene，PaodingAnalyzer

集成开发环境：MyEclipse10

系统结构：SSH（Struts2，Spring，Hibernate）

服务器程序：Tomcat7.0

客户端程序：IE6+，Chome 浏览器

Java 包名与组织结构的设计如下表所示：

图 4-1 Java 的包名与组织结构

包名	说明
blog.model	持久化类
blog.dao	DAO 接口
blog.dao.hibernate	用 Hibernate 框架实现 DAO 接口的实现类
blog.service	业务逻辑层的接口
blog.service.impl	实现了业务逻辑层的接口的实现类
blog.action	Struts2 中的 Action 类
blog.util	工具类

4.1.3 编码实现

1. 在 MyEclipse 中建立项目
2. 编写域模型代码
3. 编写持久层相关的代码
4. 编写业务逻辑层相关的代码
5. 编写表现层相关的代码

4.1.4 部署运行博客系统

部署本系统首先要安装 JDK，Tomcat 服务器；之后安装 MySQL，运行建表语句；最后在 Tomcat 上部署 blog 的 war 文件，并将系统运行的辅助文件解压到 D 盘。

博主打开浏览器，并且登录网站后，会跳转到博主的后台博客页面，获得系统管理模块、分类管理模块、文章管理模块、相册管理模块和评论模块等。

本论文核心为全文检索的两种实现，故省略各个模块的介绍。下面将分别概要介绍用费舍尔方法实现的全文检索和用 Luene 实现的全文检索。

4.2 费舍尔方法实现全文检索

费舍尔方法实现全文检索，首先要进行域模型的设计，需求分析以及要实现的功能，使用 powerdesinger 设计出如下两个表结构：

名	类型	长度	小数点	允许空值 (	
id	int	11	0	☐	🔑1
blog_id	int	11	0	☑	
entry_id	int	11	0	☑	
word	varchar	40	0	☑	
count	int	11	0	☑	

图 4-6 entry_count 表

名	类型	长度	小数点	允许空值 (	
id	int	11	0	☐	🔑1
blog_id	int	11	0	☑	
word	varchar	40	0	☑	
count	int	11	0	☑	

图 4-7 blog_count 表

这两个表分别记录了博文分词后的分词结果。entry_count 表用来统计一篇博文中每个中文分词出现在此文中多少次；blog_count 表用来统计一个博客系统中每篇博文总共的分词数。然后生成相应的建表语句，通过 Navicat 在 MySQL 数据库中生成对应的表 entry_count 和 blog_count。blog_count 表主要是用于为了以后将个人博客系统扩展为多用户博客平台时所需的功能。实际上，对于个人博客系统而言，只要一个 entry_count 表即可满足本论文的算法。

之后通过 MyEclipse 的 Hibernate 逆向工程自动生成 EntryCount.java 和 BlogCount.java。

再根据所需要的功能编写 dao 以及 service；最后在 action 中使用 service 提供的服务编写算法总的控制逻辑。

1. 费舍尔方法实现全文检索的主要实现代码如下：

```
public String search() throws IOException {
        /* 对query中文分词 */
        List<String> queryWords=new ArrayList<String>();
    //限于篇幅，此处省去中文分词的代码
        //得到所有包含query中文分词的博文Id集合
        Set<Integer> set = new HashSet<Integer>();
        for(String word:queryWords){
    List<Integer> ids=entryCountManager.getEntryIdsByQueryWord(blog.getId(), 1,
word);
              set.addAll(ids);
        }
        List<Integer> entryIds = new ArrayList<Integer>(set);
        //存放最后的概率
        List<Double> entryProbs = new ArrayList<Double>(entryIds.size());
        for(int i=0;i<entryIds.size();i++)entryProbs.add(0.0);
        double ap=0.05;//smaller is better if only little documents
        int weight=1;
        for (String word : queryWords) {
            List<Integer> counts = new ArrayList<Integer>(entryIds.size());
            List<Integer> tatols = new ArrayList<Integer>(entryIds.size());
            int allCounts = 0;// 用来计算weightedprobs，所有博文中出现这个词的次数
            List<Double> probs = new ArrayList<Double>(entryIds.size());
            double sum = 0.0;
            for (Integer id : entryIds) {
                int count = entryCountManager.wordCount(id, word);
                int tatol = entryCountManager.totalCount(id);
                counts.add(count);
                tatols.add(tatol);
                allCounts += count;
                if(count==0||tatol==0)probs.add(0.);
```

```
                else{
                    probs.add(count * 1. / tatol);
                    sum += count * 1. / tatol;
                }
            }
            List<Double> basicprobs = new ArrayList<Double>(entryIds.size());
            for (int i = 0; i < probs.size(); i++) {
                if(sum==0)basicprobs.add(0.);
                else
                    basicprobs.add(probs.get(i) / sum);
            }
            List<Double> weightedprobs = new ArrayList<Double>(entryIds.size());
            for (int i = 0; i < probs.size(); i++) {
                double wp = (weight * ap + allCounts * basicprobs.get(i))
                        / (weight + allCounts);
                weightedprobs.add(wp);
                entryProbs.set(i, entryProbs.get(i) + Math.log(wp));
            }
        }
        for (int i = 0; i < entryIds.size(); i++) {
            double p=-2.0*entryProbs.get(i);
            entryProbs.set(i, invchi2(p, 2*queryWords.size()));
            System.out.println("ID:" + entryIds.get(i) + ", prob="    +
    entryProbs.get(i) + "\t");
        }
        //对得到的每篇博文进行按概率大小排序
        ……//限于篇幅，此处省去排序的代码
        List<Entry> list=new ArrayList<Entry>();
        for(Integer id:entryIds){
            Entry e = entryManager.get(id);
            list.add(e);
        }
        pagination.setList(list);
    }
```

2. 费舍尔方法用于全文检索中的后期模型的持续学习代码如下

```
// learning from click
public String learn() throws IOException{
     ActionContext context = ActionContext.getContext();
      Map params = context.getParameters();
      String[] queries=(String[])params.get("query");
      query=queries[0];
      query=new String(query.getBytes("iso-8859-1"), "utf-8");
    // learning from click
    entry = entryManager.get(id);
    /* load执行成功后的转向页面来显示具体对象内容 */
    comments = commentManager.getListByEntryId(id);
    redirectUrl = "/entry.jsp";
    Analyzer analyzer = new net.paoding.analysis.analyzer.PaodingAnalyzer();
    StringReader reader = new StringReader(StringUtil.stripHTML(query));
    TokenStream tokenStream = analyzer.tokenStream("",  reader);
    Token t;
    t = tokenStream.next(null);
    while (t != null) {
        if(t.term()==null) continue;
        entryCountManager.increase(Const.BLOG_ID,  entry.getId(), t.term());
        t = tokenStream.next(t);
    }
}
```

4.3 Lucene实现全文检索

使用 lucene 来实现全文检索是不需要在数据库中存储用于全文检索的索引及分词的。Lucene 会自动在服务器端指定的位置创建索引文件。使用 lucene 来实现全文检索分两个步骤：首先要创建索引，对待检索的博文进行中文分词并建立索引文件；然后通过索引查询相关博文。

1. 用 lucene 实现全文检索创建索引的代码如下：

```
public void luceneIndex(){
    AnnotationConfiguration config = new AnnotationConfiguration().configure();
```

```
    SessionFactory sf = config.configure().buildSessionFactory();
    Session session = sf.openSession();
    FullTextSession ftSession = Search.getFullTextSession(session);
    ftSession.getTransaction().begin();

    List<Entry> eList = session.createQuery("from Entry order by id desc").list();
    for (Entry e : eList) {
        Entry en = e;//session.save(e);//if not:The instance Not associated with
   this session
        ftSession.index(en);
        System.out.println(e.getTitle()+" is DONE!");
    }
    ftSession.getTransaction().commit();
    System.out.println("lucene index is DONE!");
}
```

2. 用 lucene 实现全文检索查询相关博文的代码如下：

```
/*    search using lucene    */
public String luceneSearch() throws Exception {
    /* 对query中文分词 */
    List<String> queryWords=new ArrayList<String>();
    Analyzer analyzer = new net.paoding.analysis.analyzer.PaodingAnalyzer();
    StringReader reader = new StringReader(StringUtil.stripHTML(query));
    TokenStream tokenStream = analyzer.tokenStream("",  reader);
    Token t;
    t = tokenStream.next(null);/*-try-catch-*/
    while (t != null) {
        System.out.print(t.term() + "\t");
        queryWords.add(t.term());
        if (t.term() == null) continue;
        t = tokenStream.next(t);/*-try-catch-*/
    }
    AnnotationConfiguration config = new AnnotationConfiguration().configure();
    SessionFactory sf = config.configure().buildSessionFactory();
    Session session = sf.openSession();
    FullTextSession ftSession = Search.getFullTextSession(session);
```

```
    ftSession.getTransaction().begin();
        String[] queries= new String[queryWords.size()];
        String[] fields = new String[queryWords.size()];
      BooleanClause.Occur[] clauses = new BooleanClause.Occur[queryWords.size()];
        for(int i=0;i<queryWords.size();i++){
         queries[i]= queryWords.get(i);
         fields[i] = "content";
         clauses[i]= BooleanClause.Occur.SHOULD;
        }
 org.apache.lucene.search.Query luceneQuery =
MultiFieldQueryParser.parse(queries, fields, clauses, new
net.paoding.analysis.analyzer.PaodingAnalyzer());
  Query qry = ftSession.createFullTextQuery(luceneQuery, Entry.class);
    List<Entry> es = qry.list();
    ftSession.getTransaction().commit();
    Integer pageSize = blogManager.get(Const.BLOG_ID).getBackendPageSize();
    pagination = entryManager.getPaginationByBlogId(Const.BLOG_ID, null,
 pageSize, p);
    pagination.setList(es);
    return SUCCESS;
}
```

4.4 两种实现方案的对比

为了实现两种方案的对比，本论文使用台湾作家刘墉的三本励志书《超越自己》、《肯定自己》、《创造自己》作为数据集，并且将“超越自己”作为查询语句以对比两种方案。使用庖丁解牛分词器对“超越自己”进行分词后的结果为“超越”和“自己”两个查询词组。本论文使用下面的 SQL 语句来查看有哪些博文同时含有“超越”和“自己”这两个词组，查询结果如图 4-8 所示：

```
SELECT * FROM entry
WHERE entry.id IN
(
SELECT ec1.entry_id FROM entry_count ec1, entry_count ec2
WHERE ec1.word = "超越" AND ec2.word = "自己" AND ec1.entry_id =
ec2.entry_id
)
```

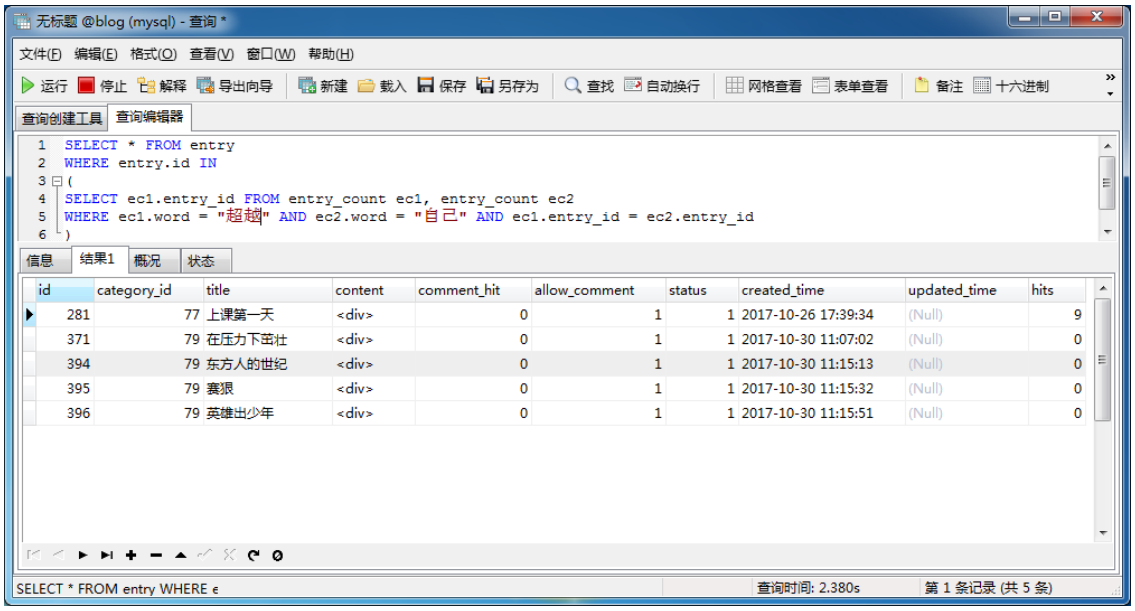


图 4-8 同时含有“超越”和“自己”的博文

从图中可以发现一共有 5 篇博文同时含有“超越”与“自己”两个词组。进一步查询发现，含有词组“超越”的博文一共有 5 篇；含有词组“自己”的博文一共有 101 篇。

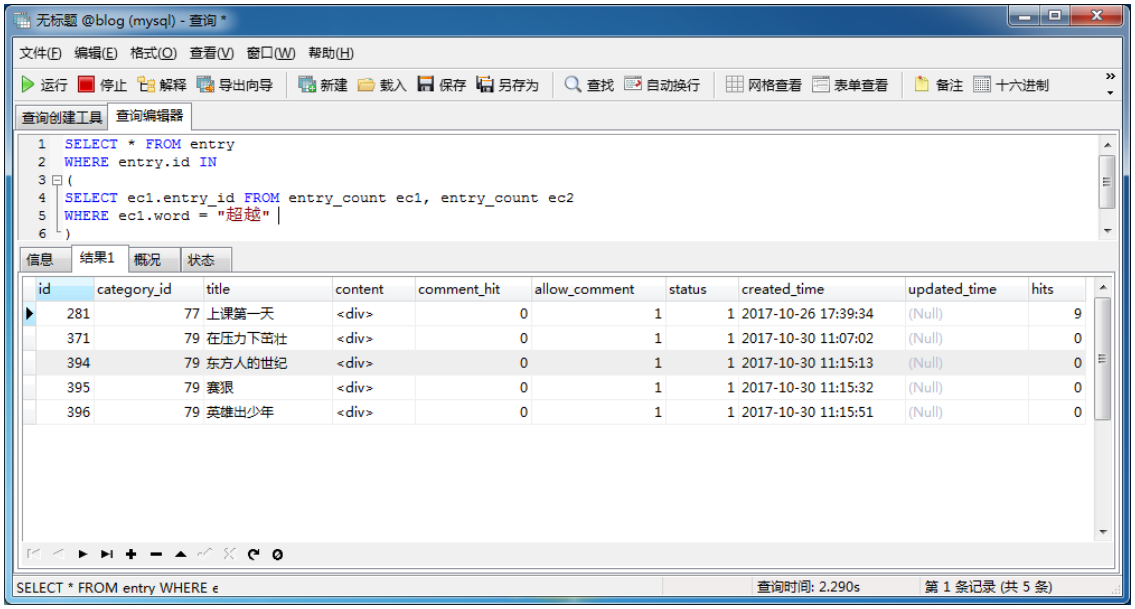


图 4-9 含有“超越”词组的博文

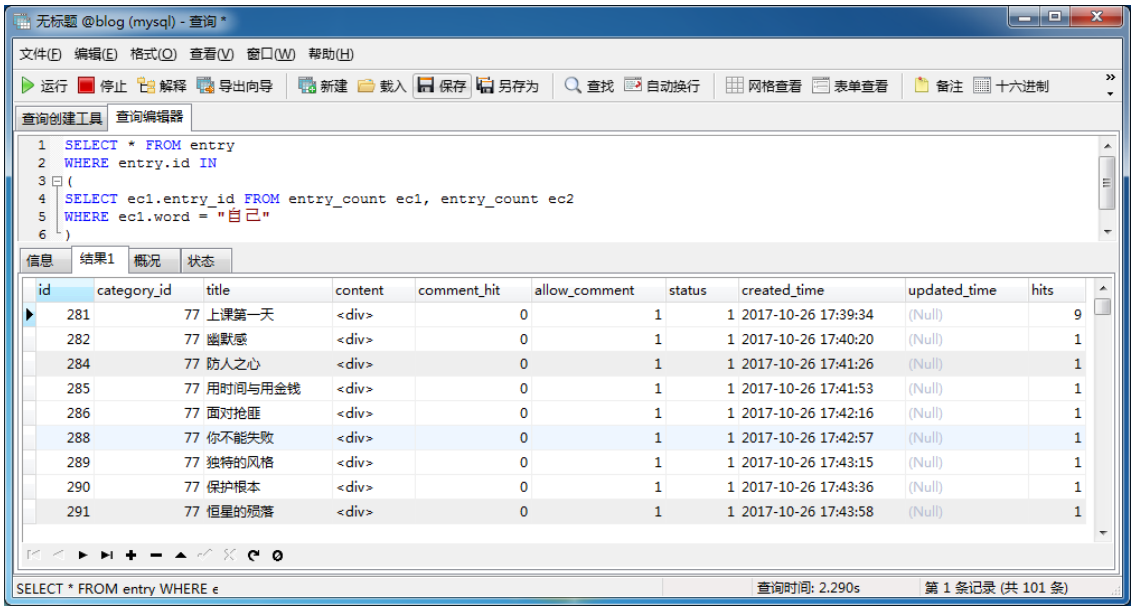


图 4-10 含有“自己”词组的博文

1. 用 lucene 实现的方案查询结果如下图所示。从中可以看到，lucene 实现的查询方法返回的结果前五条博文都是同时含有“超越”与“自己”的博文，符合用户的查询要求。但是，lucene 实现的查询结果是固定不能改变的。

图 4-11 用 lucene 实现的方案查询结果

2. 本论文设计的算法的查询结果如下图所示。从中可以看到，查询返回的结果中前五条博文有四条记录是同时含有“超越”与“自己”的博文，还有另一同时含有“超越”

与“自己”的博文在查询结果中排到了第七位。根据算法的逻辑，这是因为有两篇博文中“自己”的词频数较大，最后计算出的概率比同时含有“超越”与“自己”的那篇博文大，从而将该篇博文挤出了前五的排序结果。不过，这样的查询结果也符合用户的查询要求，因为这两篇博文中含有较大的“自己”的词频，也是用户查询要求下的潜在想要的查询结果。

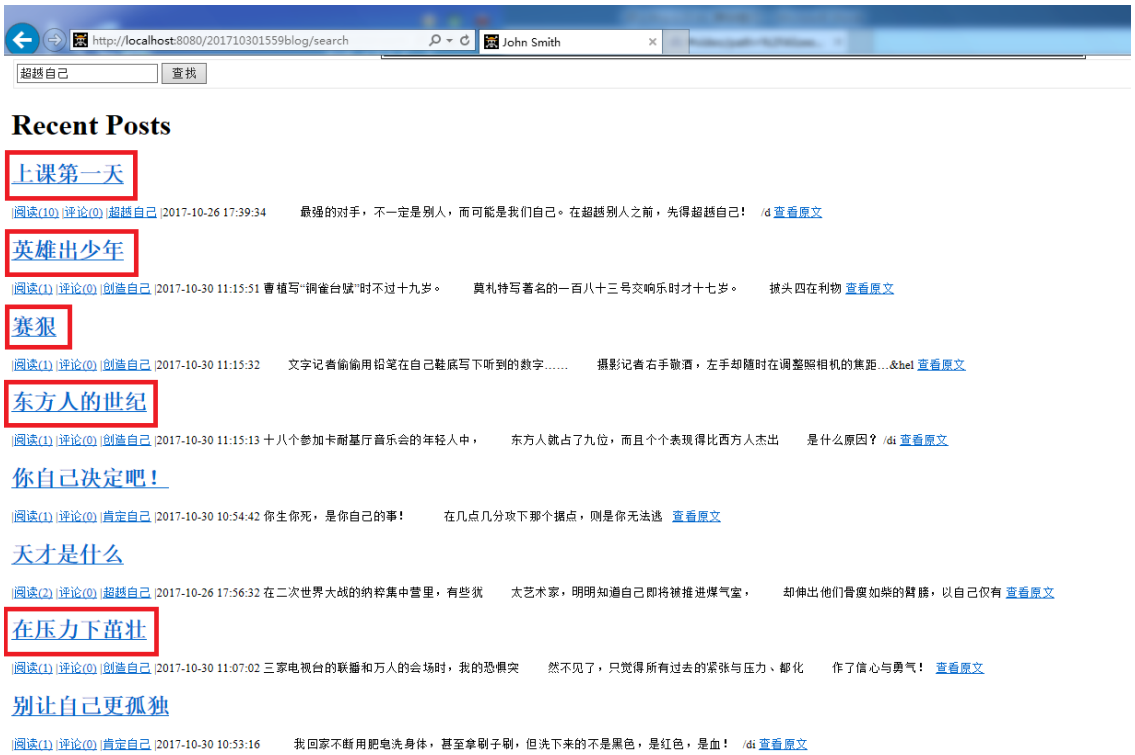


图 4-12 本论文设计的算法的查询结果

此外，本论文实现的算法是可以进行持续学习的，用户的查询结果随着用户的点击选择而不断改变的。

两次连续查询点击博文“东方人的世纪”后，该篇博文由查询结果中的第四位上升到第二位。

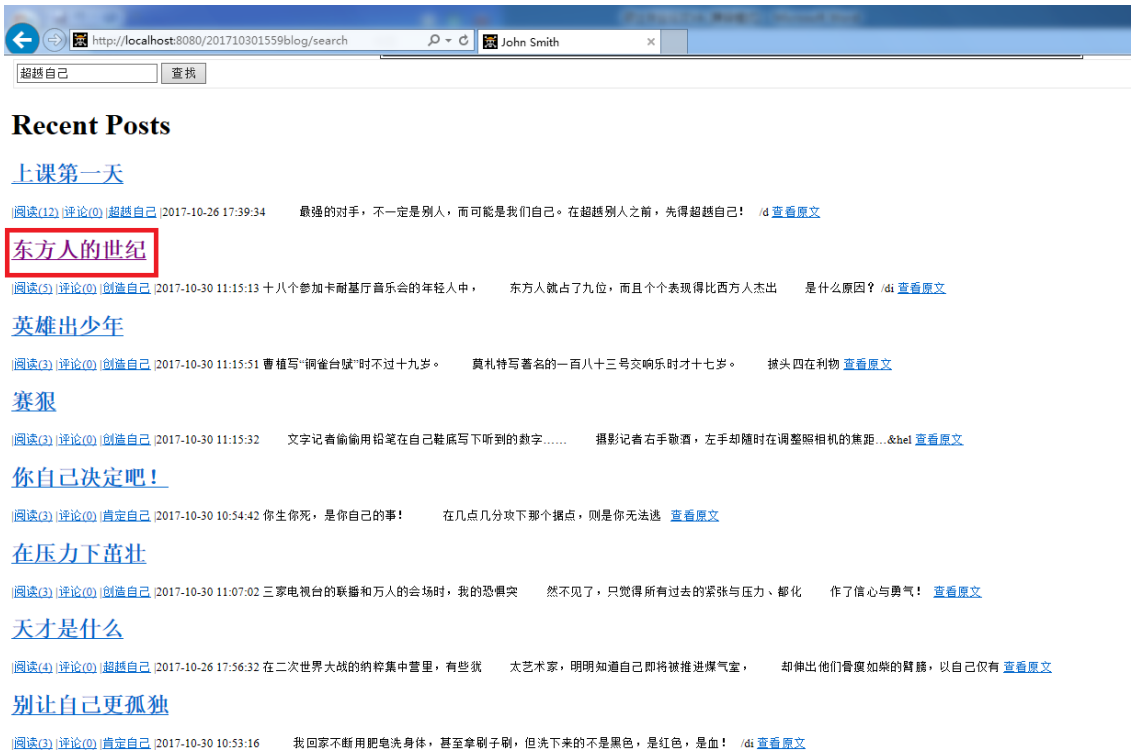


图 4-13 第两次查询博文“东方人的世纪”

第三次点击“东方人的世纪”后，该篇博文在查询结果中由第二位升到第一位。

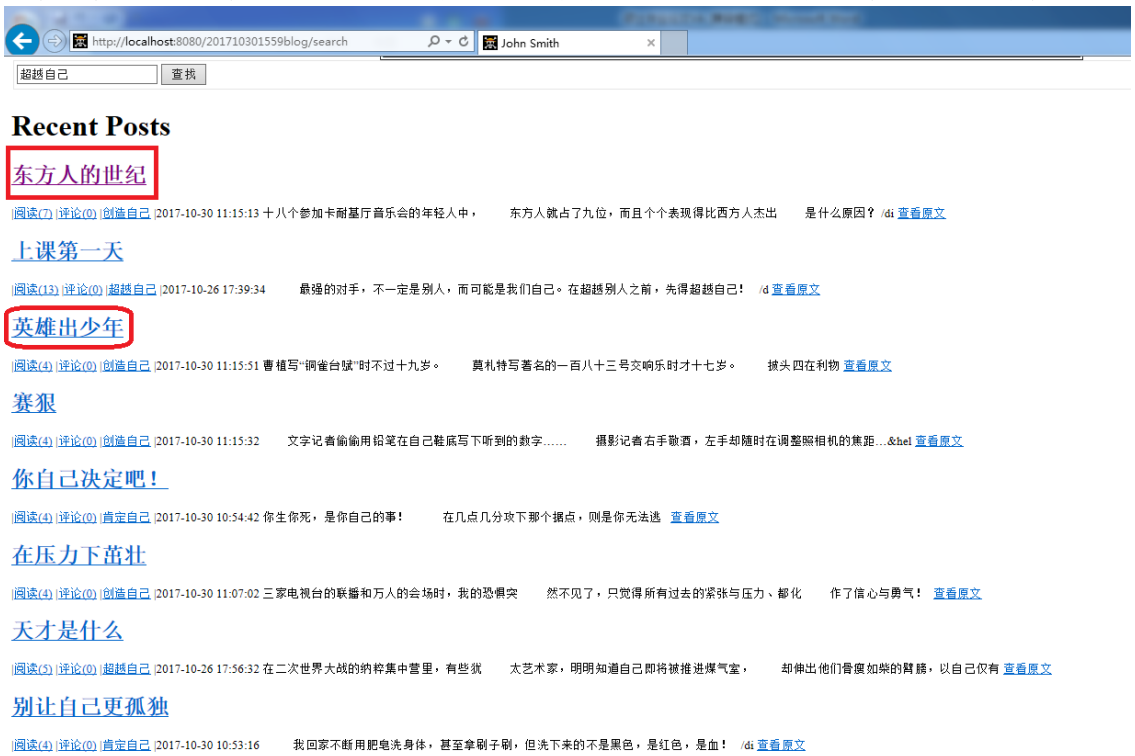


图 4-14 第三次点击“东方人的世纪”

之后对博文“英雄出少年”进行一次持续学习，该博文由第三位上升到第二位。

图 4-15 对博文“英雄出少年”进行一次持续学习

随着持续学习次数的增多，进行持续学习的博文一点一点的逐渐提升查询结果中的排名。这不仅反映了算法的稳定性与正确性，也防止了查询结果的反复颠簸。通过设置持续学习的学习率可以进一步控制算法的稳定性。

如果在算法最后计算出的后验概率中加入其它的参数及其权重，那么用户的查询结果将不仅仅随着用户的点击而改变，同时也会睡着每篇博文的浏览量、评论数、发表时间和更新时间等其它因素而改变。这将会给用户带来更好的体验。

4.5 本章小结

本章重点介绍了如何使用 SSH 框架构建博客系统，实现了本论文的核心算法。在该算法实现的系统中：用户可以在查询输入框中输入想查询信息的关键字，并可以得到系统中已有的博文列表。博文列表会根据输入的关键字的相关度而进行排序并返回；用户在得到查询返回的博文列表后，通过选择某一博文可以逐渐改变该博文在查询结果中的顺序。

第五章 结论与展望

本论文在全面研究全文检索技术和全面介绍 lucene 相关技术的基础上，设计实现了一个博客系统，并在此系统上编程实现了两种全文检索方案。论文中完成的主要工作包括以下几点：

1. 通过分析应用于文本分类的朴素贝叶斯模型，本文提出了一个可以用于个人博客系统的全文检索方案。该方案可以随着用户选择某篇查询后的博文而进行持续学习，并且通过多次持续学习而逐渐改善查询结果，最终使查询结果更符合用户的查询要求。

2. 在使用 SSH 框架构建的博客系统上实现了两种全文检索方案：本论文提出的全文检索方案和用 lucene 实现的全文检索方案。本论文通过编程实现对比了两种全文检索的查询结果。通过比较可以得知，lucene 实现的全文检索有更好的准确性但是查询结果是固定不变的，没有智能性；本论文实现的全文检索方案在检索的准确性有待提高，在对查询结果的博文排序上有较大的灵活性，可以随着用户查询点击量的增加而逐渐改善查询结果，使其更符合用户的查询要求，并增加博客系统的智能性，提高用户的体验度。

全文检索系统是一个非常复杂的系统，其中的每个环节都会对整体的性能产生影响，为此对于全文检索系统的改善不能只停留在对中文分词的改进上。后续的工作可以从以下方面进行开展：

1. 本论文的算法在查询结果的排序上虽然灵活度很高，但是时间效率却不尽如人意。后续的研究工作应着眼于如何提高时间效率上，可以采用改进数据库表结构，优化算法等方法。

2. 在使用 SSH 框架对于检索结果的排序技术需要进行改进。对于检索结果，一般都会利用所设定的评分算法对其进行排序。lucene 自带的排序方法虽然比本论文提出的算法效果好一些，但并不完美。lucene 自带的排序方法所生成的查询结果并不能随着用户查询次数的增加而改善；而本论文设计的算法可以随着用户的多次查询后该算法生成的模型会逐渐完善。后续工作可以合并这两种全文检索方案，即在 lucene 全文检索方案上吸取本论文设计算法的可取之处，从而设计一个同时拥有两种全文检索方案长处的算法。

参考文献

[1] Michael McCandless，Erik Hatcher，Otis Gospondnetic. Lucene in Action[M]. Manning Publications，2010.
[2] P.A Mitkas，P.S Guilfoyle. An Optical System for Full-Text Search. SIGIR89: Proceedings of the 12th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval，2005.
[3] Linwood J. Give Your Web Site Search Engine Using Lucene[Z]. http:// builder.com， 2003.
[4] Laurence Hirsch，Robin Hirsch，Masoud Saeedi. Evolving Lucene Search Queries for Text Classification. Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, 2007.
[5] Toby Segaran. Programming Collective Intelligence[M]. O'Reilly Media，2007.
[6] Pang-Ning Tan，Michael Steinbach，Vipin Kumar. Introduction to data mining[M]. Pearson Addison Wesley，2006.
[7] Ian H. Witten，Eibe Frank. Data Mining :Practical Machine Learning Tools and Techniques[M]. Morgan Kaufmann，2016.
[8] Peter Harrington. Machine Learning in Action[M]. Manning Publications，2012.
[9] Tom M. Mitchell. Machine Learning[M]. McGraw-Hill Education，1997.
[10] Jure Leskovec, Anand Rajaraman, Jeffrey David Ullman. Mining of Massive Datasets[M]. Cambridge University Press，2014.
[11] 李永春，丁华福.Lucene 的全文检索的研究与应用[J]. 计算机技术与发展，2010(02).
[12] 高配芝.基于相关词识别的数据库全文检索研究[D]. 山东：山东大学，2014.
[13] 张校乾.基于 Lucene 的全文检索系统的研究与应用[D]. 大连：大连理工大学，2005.
[14] 侯林.基于 MVC 设计模式的博客系统的设计与实现[D]. 大连：大连理工大学，2011.
[15] 郎小伟，王申康.基于 Lucene 的全文检索系统研究与开发[J]. 计算机工程，2006(04).
[16] 莫映，王开福译. 集体智慧编程[M]. 电子工业出版社，2009.
[17] 范蕾.基于 Lucene 的全文检索系统的设计与实现[D].厦门：厦门大学，2014.
[18] 李博.博客检索的关键技术研究[D].哈尔滨：哈尔滨工业大学，2009.
[19] 韩春晓.中文期刊个性化搜索引擎的设计与实现[D].哈尔滨：哈尔滨工业大学，2014.
[20] 江毅铭.专业搜索引擎索引技术的研究与实现[D].北京：北京化工大学，2005.
[21] 刘伟，张利国. Hibernate 开发与实战[M]. 电子工业出版社，2009.
[22] 何发胜.基于 SSH 模式架构的博客系统的设计与实现[D]. 湖南：湖南大学，2012.
[23] 李锐，李鹏，曲亚东，王斌译. 机器学习实战[M]. 人民邮电出版社，2013.
[24] 阿曼. 朴素贝叶斯分类算法的研究与应用[D]. 大连：大连理工大学，2014.
[25] 张利国，刘伟. Java SE 应用程序设计[M]. 科学出版社，2008.
[26] 刘伟，张利国. Java Web 开发与实战[M]. 科学出版社，2008.
[27] 董琳，邱泉，于晓峰译. 数据挖掘 实用机器学习技术[M]. 机械工业出版社，2006.
[28] 杜婷.基于属性选择的朴素贝叶斯分类研究与应用[D]. 安徽：中国科学技术大学，2016.
[29] 曹强.基于 Lucene 的 Web 站点站内全文检索系统的设计与实现[J]. 图书情报工作，2007(09).
[30] 陈洪猛.全文检索技术的研究与实现[D]. 北京：北京工业大学，2008.

发表论文和科研情况说明

无

致　　谢

感谢天津理工大学为我提供的一切学习资源，感谢天津理工大学让我不用啃老就获得了硕士学位。

感谢硕导岳廷海老师为我读研期间提供的帮助与指导，感谢岳老师对我独特个性的宽容与接纳；如果换做是其他的老师，我可能在硕士期间不会像现在这样顺利；感谢张德干老师两次为我推荐导师，或许辜负了张老师的一番用意，最后没能跟他学网络。

感谢自己多年的努力，高中逃课辍学浪费掉的迷茫的四年时光、本科毕业后又接连三次报考交大、浙大研究生未果而浪费的三年时光，等完成最终的计划获得自由以后，我必会用三十年的时间来弥补这些遗憾洗雪这些耻辱，如果我能活到那个时候的话。

2018-01-14 Sunday 11:55 a.m. GMT+8 天理图书馆 2D 区 A036 座